AutoRestTest

An AI-Powered Platform for Automated REST API Testing

SE 801
Software Project Lab 3 Final Report

Submitted by:

Mohammad Ismail Hossain
Exam Roll: 2322122
BSSE Roll: 1433
Institute of Information Technology
University of Dhaka

Supervised by:

Toukir Ahammed
Lecturer
Institute of Information Technology
University of Dhaka





Submission Date: 10 October, 2026

Letter of Transmittal

10 October, 2026

BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Final Report Submission - **AutoRestTest**: An AI-Powered Platform for Automated REST API Testing

Sir,

I am submitting the final report for our Software Project Lab 3, **AutoRestTest**: An AI-Powered Platform for Automated REST API Testing. This report comprehensively covers the project's technical details, including requirement analysis, design details, testing results, user manual. While I've tried to do my best, I welcome your feedback for improvement.

Thank you for your consideration.

Sincerely,

Mohammad Ismail Hossain
Exam Roll: 2322122
BSSE Roll: 1433
Institute of Information Technology
University of Dhaka

08/10/2026

Supervisor's Signature

Acknowledgment

I am very thankful to the **Institute of Information Technology** for granting me the opportunity to work on the Software Project Lab III, **AutoRestTest**: An AI-Powered Platform for Automated REST API Testing.

I would like to express my gratitude to my supervisor, **Toukir Ahammed, Lecturer, Institute of Information Technology, University of Dhaka**, for his invaluable guidance and support throughout the project. His expertise and insights were instrumental in shaping the direction and success of this work.

Finally, I would like to thank my classmates, whose help and timely feedback provided valuable perspectives that contributed to the project's development.

Abstract

AutoRestTest is an intelligent, collaborative SaaS platform designed to automate and optimize the functional testing of REST APIs. As modern web services increasingly rely on complex REST APIs, ensuring their correctness has become a critical challenge. Traditional automated tools often fail to explore interdependent operations effectively, resulting in low coverage and weak fault detection, while manual testing remains time-consuming, error-prone, and difficult to scale alongside continuous development cycles.

Leveraging advances in artificial intelligence, AutoRestTest integrates Multi-Agent Reinforcement Learning (MARL), a Semantic Property Dependency Graph (SPDG), and Large Language Models (LLMs) to maximize endpoint coverage and uncover genuine server-side faults. The system automatically ingests OpenAPI specifications or generates them directly from uploaded source code and uses four specialized agents to navigate the API, resolve complex data dependencies, and inject context-aware parameter values without requiring developers to write manual test scripts.

Beyond robust test execution, AutoRestTest enhances the developer experience by offering plain-language, AI-powered failure explanations. When the testing engine triggers an unexpected internal server error (500 status code), the platform's LLM analyzes the specific request and response to clearly explain what went wrong and which request triggered it, drastically reducing the time spent debugging.

The platform also supports long-term quality assurance by allowing development teams to save, reuse, and track automated test suites over time within a centralized, role-based collaborative workspace. By effectively addressing the limitations of existing black-box testing tools and significantly outperforming them in both endpoint coverage and fault detection, AutoRestTest serves as a highly scalable, reliable solution that reduces manual effort and improves API quality for modern development teams.

Table of Contents

Project overview **7**
1.1 Project Title 7
1.2 Problem Statement 7
1.3 Objectives 7
1.4 Scope 7
Requirements Analysis **8**
2.1 Quality Function Deployment (QFD) 8
2.1.1 Normal Requirements 8
2.1.2 Expected Requirements 9
2.1.3 Exciting Requirements 9
Database Design **10**
3.1 ER Diagram 10
3.2 Database schema 10
Component-Level Design **13**
4.1 Design Classes and Problem Domain Mapping 13
4.2 Class-Responsibility-Collaborator (CRC) Cards 15
4.2 Persistent Data Sources 18
Interface Design **19**
5.1 User Interface Architecture 19
5.2 UI Events & State Transitions 20
5.2 State Transition Diagram 21
Testing **23**
6.1 Testing Approach & Strategy 23
6.2 Item Pass/Fail Criteria 23
6.3 Risks and Contingencies 24
6.4 Unit Test Summary 24
6.5 Detailed Test Cases (Black box test) 26
Module 1: Authentication system 26
Module 2: Project Management 30
Module 3: API Specification Management 31
Module 4: Test Suit Management 36
Module 5: Test Execution 37
Module 6: Test Results & Reports 39
Module 7: Team Collaboration 41
Real-World Evaluation **43**
7.1 Overview 43
7.1 Evaluation Methodology 43
7.3 Evaluation Summary 44

User Manual **45**
8.1 Introduction 45
8.2 Installation 45
8.3 Getting Started 47
8.4 Account Creation 47
8.5 Sign In 49
8.6 Application Layout 51
8.7 Projects 51
8.8 Project Roles and Permissions 53
8.9 Project Overview 53
8.10 API Specification 54
8.11 API Endpoints 56
8.12 Dependency Graph 57
8.13 Test Suites 58
8.14 Captured Requests and Live Request Runs 61
8.15 Team Management 61
8.16 Account Settings 63
8.17 Logging Out 65
8.18 Frequently Asked Questions 65
Conclusion **66**
References **67**

List of Figures

Figure 3.1: Entity Relationship Diagram 7

Figure 5.1: State Transition for Authentication and User Identity Lifecycle 18
Figure 5.2: State Transition for Project Collaboration & Access Control 18
Figure 5.3: State Transition for Automated API Spec generation 19
Figure 5.4: State Transition for Automated Test Suit Execution 19

Figure 8.1: AutoRestTest Landing Page 42
Figure 8.2: Account register page 43
Figure 8.3: Email verification page 44
Figure 8.4: Login page 45
Figure 8.5: Reset password page 46
Figure 8.6: Navbar 47
Figure 8.7: Homepage dashboard page 47
Figure 8.8: Create new project form 48
Figure 8.9: Project overview tab 49
Figure 8.10: API specification upload view 50
Figure 8.11: API endpoints view 52

Figure 8.12: Semantic dependency graph view 53
Figure 8.13: Test suites view 54
Figure 8.14: Timeline Gantt Chart 41
Figure 8.15: Test run report view 55
Figure 8.16: Captured request logs view 56
Figure 8.17: Team members view 57
Figure 8.18: Send project invitation view 58
Figure 8.19: Pending invitations list 58
Figure 8.20: User settings page 59
Figure 8.21: LLM settings page 60

List of Tables

Table 3.1: Database schema 7

Table 4.1: Problem Domain to Design Class Mapping 9
Table 4.2: AuthService CRC Card 12
Table 4.3: TokenService CRC Card 12
Table 4.4: ProjectService CRC Card 13
Table 4.5: ProjectAccessService CRC Card 13
Table 4.6: SpecsService & SpecificationParser CRC Card 13
Table 4.7: SpecGenerationService CRC Card 14
Table 4.8: TestSuiteService & JobManager CRC Card 14
Table 4.9: MARL Engine CRC Card 14
Table 4.10: GraphService CRC Card 15
Table 4.11: ReportService CRC Card 15
Table 4.12: Persistent Data Sources 15

Table 5.1: UI Objects and Actions 15
Table 5.2: Key UI Events and State Changes 16

Table 6.1: Risks and Contingencies in testing 20
Table 6.2: Unit Test Summary 20
Table 6.3: Black Box test summary 22

Table 7.1: Real-World Evaluation Summary 40

Chapter 1

Project overview

1.1 Project Title

AutoRestTest: An AI-Powered Platform for Automated REST API Testing

1.2 Problem Statement

As modern web services increasingly rely on REST APIs, ensuring their correctness is critical. As APIs grow in size and complexity, testing them becomes progressively harder because of large input spaces, interdependent operations, and the need for valid request sequences. Traditional automated testing tools often fail to explore these dependencies effectively, leading to low coverage, missed edge cases, and weak fault detection, while manual testing is time-consuming, repetitive, error-prone, and unable to scale with continuous development. Recent research such as AutoRestTest [2] shows that dependency-aware, reinforcement-learning and LLM-powered test generation can substantially improve coverage and fault detection for REST APIs. At the same time, development teams need a centralized, collaborative environment in which specifications, test suites, execution results, and reports can be stored, shared, and reused over time. Existing solutions rarely combine an intelligent testing engine with such a platform, and this gap is the problem this project addresses.

1.3 Objectives

The main objectives of this project are as follows:

- To design and develop an AI-powered, collaborative SaaS platform for automated REST API testing, accessible through a centralized web dashboard.
- To integrate an intelligent testing engine that uses dependency-aware, reinforcement-learning and LLM-based request generation, inspired by the AutoRestTest [2] approach, to maximize coverage and fault detection.
- To allow users to upload OpenAPI specifications, or generate them with AI-assisted analysis, and to extract their endpoints automatically, inspired by the OOPS [3] approach .
- To automatically generate, save, and reuse test suites for each project.
- To provide one-click, asynchronous test execution with real-time status updates and persistent execution history.
- To present clear results and analytics, including status-code distribution, endpoint coverage, and server failures, together with LLM-based request & failure explanations and exportable PDF/CSV reports.
- To enable secure team collaboration through project sharing and role-based access control.

1.4 Scope

The scope of this project defines the boundaries of the platform and the extent of the functionality it delivers. AutoRestTest is a web-based SaaS application for the functional

testing of RESTful web services described by OpenAPI 3.x specifications. Within this boundary it supports the complete testing workflow through a single centralized dashboard, from user and project management to specification handling, AI-driven test generation and execution, result analysis and reporting, and multi-user collaboration, so that development teams can automate REST API testing and reduce manual effort.

Certain areas are intentionally outside the scope of this project. The platform targets REST APIs only and does not address SOAP, GraphQL, or gRPC services, nor specification formats other than OpenAPI 3.x. It focuses on functional and fault-detection testing rather than load, stress, or performance testing, and it does not modify, deploy, or manage the systems under test. The platform further assumes that the target API is reachable over the network and that any credentials required to access protected endpoints are provided by the user.

Chapter 2

Requirements Analysis

2.1 Quality Function Deployment (QFD)

2.1.1 Normal Requirements

Explicitly requested core features; user satisfaction is proportional to how well they are met.

Functional Requirements

- ☐ **FR-N1 User Authentication:** register, log in, and maintain secure sessions.
- ☐ **FR-N2 Project Management:** create, open, rename, and delete API testing projects from a central dashboard.
- ☐ **FR-N3 Specification Upload:** upload an OpenAPI 3.0 file in .yaml or .json format; the system parses it and lists the extracted endpoints.
- ☐ **FR-N4 Test Execution:** run automated requests against the target API from a specification and collect the responses.
- ☐ **FR-N5 Result Viewing:** view each run's outcome per endpoint (method, request, response, status code, pass/fail).
- ☐ **FR-N6 Completion Notifications:** notify the user when a run completes, fails, or errors.

Non-Functional Requirements

- ☐ **NFR-N1 Usability:** the whole workflow is driven from one intuitive web dashboard, usable without special training.
- ☐ **NFR-N2 Responsiveness:** routine interactions (dashboard, navigation) load within a couple of seconds.

2.1.2 Expected Requirements

Implicitly assumed; their absence causes dissatisfaction even though users rarely ask for them by name.

Functional Requirements

- ☐ **FR-E1 Automatic Test Suite Generation:** automatically generate a comprehensive, dependency-aware suite of test cases from the specification.
- ☐ **FR-E2 Save & Reuse Suites:** persist named test suites for re-execution without reconfiguration.
- ☐ **FR-E3 Execution History:** keep every past run with its timestamp and outcome for later review.
- ☐ **FR-E4 Reports & Analytics:** per-run summary of HTTP status-code distribution, coverage, and server failures.
- ☐ **FR-E5 Export Reports:** download a run's results as PDF or CSV.

- ☐ **FR-E6 Team Collaboration:** invite collaborators to a project and assign roles (admin, tester, viewer).

Non-Functional Requirements

- ☐ **NFR-E1 Security & Data Privacy:** passwords hashed, routes JWT-protected, and each user's and project's data isolated via role-based access control.
- ☐ **NFR-E2 Reliability:** long-running executions run asynchronously without blocking the UI and report status accurately.
- ☐ **NFR-E3 Data Persistence:** specifications, suites, runs, and results are stored durably and remain available across sessions.

2.1.3 Exciting Requirements

Delighters beyond expectation; their absence isn't penalised, but their presence differentiates the platform.

Functional Requirements

- ☐ **FR-X1 One-Click End-to-End Testing:** a single action runs the entire pipeline — read spec, generate tests, execute, and produce a report.
- ☐ **FR-X2 AI Failure Explanation:** LLM-generated, plain-language explanation of what caused a failure and which request triggered it.
- ☐ **FR-X3 AI Specification Generation:** automatically generate an OpenAPI specification from uploaded source code.
- ☐ **FR-X4 SPDG visualization:** Interactive visualization of the semantic property dependency graph (SPDG) of the API endpoints.

Non-Functional Requirements

- ☐ **NFR-X1 Extensibility:** the AI testing engine is deployed as an independent service, so its generation strategy can evolve without changing the web platform.

Chapter 3

Database Design

Database modeling describes how the platform's persistent data is structured and related. AutoRestTest uses a relational schema on **PostgreSQL**, accessed through the **Prisma** ORM. The model comprises nine entities and four enumerated types.

3.1 ER Diagram

Entity Relationships:

- A **User** can own many **Projects**.
- A **User** can belong to many **Projects** through **ProjectMember**, a many-to-many association that carries the member's role (each row pairs one user with one project).
- A **Project** can have many **ProjectInvitations**, and each invitation is sent by one **User**.
- A **Project** has at most one **ApiSpecification**.
- A **Project** contains many **Endpoints**.
- A **Project** can have many **TestSuites**, and each **TestSuite** is triggered by one **User**.
- A **TestSuite** produces many **TestCases**, and each **TestCase** tests one **Endpoint**.
- A **TestSuite** captures many **RequestLogs**, and each **RequestLog** maps to at most one **Endpoint**.

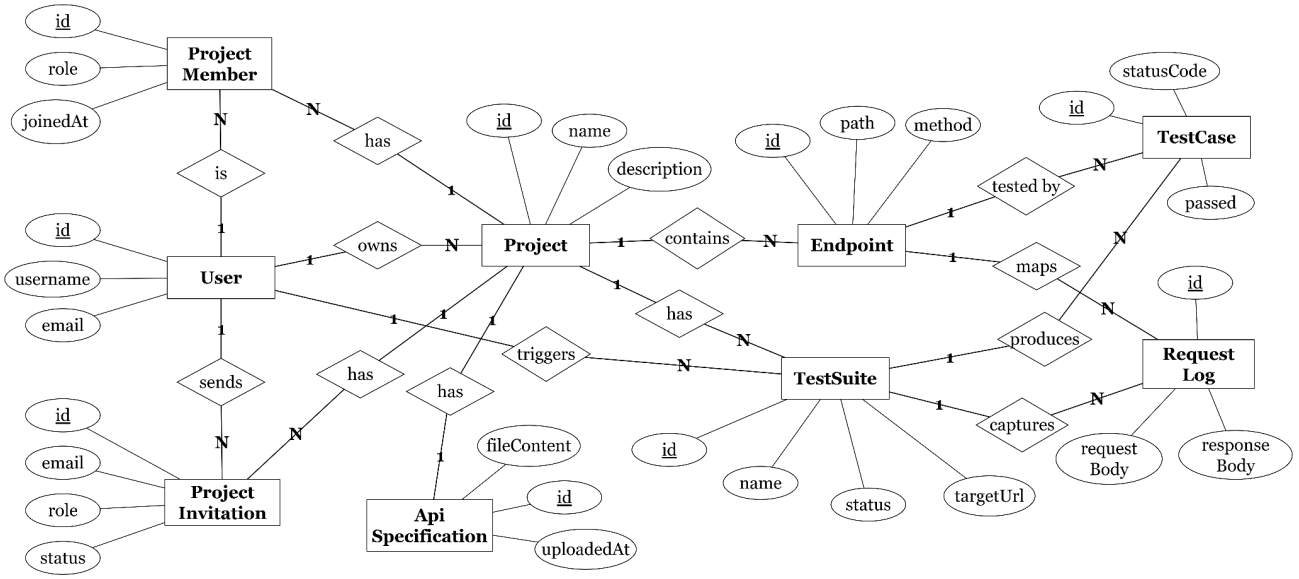


Figure 3.1: Entity Relationship Diagram

3.2 Database schema

PK = primary key, FK = foreign key, → = references. All identifiers are UUID strings.

Table – 3.1: Database schema

Schema	Attribute	Type
User	id (PK)	TEXT (UUID)
	username	TEXT, UNIQUE
	email	TEXT, UNIQUE
	password	TEXT (bcrypt hash)
	createdAt	TIMESTAMP
	updatedAt	TIMESTAMP
Project	id (PK)	TEXT (UUID)
	name	TEXT
	description	TEXT (nullable)
	ownerId (FK)	TEXT → User
	createdAt	TIMESTAMP
	updatedAt	TIMESTAMP
ProjectMember	id (PK)	TEXT (UUID)
	projectId (FK)	TEXT → Project
	userId (FK)	TEXT → User
	role	ENUM(admin, tester, viewer)
	joinedAt	TIMESTAMP
ProjectInvitation	id (PK)	TEXT (UUID)
	projectId (FK)	TEXT → Project
	email	TEXT
	role	ENUM(admin, tester, viewer)
	token	TEXT, UNIQUE
	status	ENUM(pending, accepted, declined, expired)
	invitedById (FK)	TEXT → User
	expiresAt	TIMESTAMP
	createdAt	TIMESTAMP
ApiSpecification	id (PK)	TEXT (UUID)
	projectId (FK)	TEXT → Project, UNIQUE
	fileName	TEXT
	fileContent	TEXT (raw OpenAPI 3.0)
	generatedByAI	BOOLEAN
	uploadedAt	TIMESTAMP
Endpoint	id (PK)	TEXT (UUID)
	projectId (FK)	TEXT → Project
	path	TEXT
	method	ENUM(GET, POST, PUT, PATCH, DELETE)

Schema	Attribute	Type
	description	TEXT (nullable)
	addedManually	BOOLEAN
	createdAt	TIMESTAMP
TestSuite	id (PK)	TEXT (UUID)
	projectId (FK)	TEXT → Project
	name	TEXT (nullable)
	status	ENUM(pending, running, completed, failed)
	jobId	TEXT (nullable)
	targetUrl	TEXT
	timeBudget	INTEGER
	mutationRate	DOUBLE PRECISION
	totalEndpoints	INTEGER
	coveredEndpoints	INTEGER
	totalTestCases	INTEGER
	passedTestCases	INTEGER
	failedTestCases	INTEGER
	triggeredById (FK)	TEXT → User
	startedAt	TIMESTAMP (nullable)
	completedAt	TIMESTAMP (nullable)
	createdAt	TIMESTAMP
TestCase	id (PK)	TEXT (UUID)
	testSuiteId (FK)	TEXT → TestSuite
	endpointId (FK)	TEXT → Endpoint
	requestBody	JSONB (nullable)
	requestHeaders	JSONB (nullable)
	statusCode	INTEGER (nullable)
	responseBody	JSONB (nullable)
	responseTimeMs	INTEGER (nullable)
	passed	BOOLEAN
	failureExplanation	TEXT (nullable)
	createdAt	TIMESTAMP
RequestLog	id (PK)	TEXT (UUID)
	testSuiteId (FK)	TEXT → TestSuite
	endpointId (FK)	TEXT → Endpoint (nullable)
	seq	INTEGER
	method	TEXT
	path	TEXT
	url	TEXT
	statusCode	INTEGER (nullable)

Schema	Attribute	Type
	durationMs	INTEGER (nullable)
	requestHeaders	JSONB (nullable)
	requestBody	TEXT (nullable)
	requestTruncated	BOOLEAN
	responseHeaders	JSONB (nullable)
	responseBody	TEXT (nullable)
	responseTruncated	BOOLEAN
	createdAt	TIMESTAMP

Chapter 4

Component-Level Design

4.1 Design Classes and Problem Domain Mapping

AutoRestTest is organized into modular backend services, database models, and intelligent engine components, each addressing a distinct problem domain in automated REST API testing. The table below illustrates how the core problem domain concepts map directly to their exact design classes in the codebase and their respective system responsibilities:

Table 4.1: Problem Domain to Design Class Mapping

Domain Concept	**Design Class**	**Responsibilities**
User Accounts	User model + AuthService	User registration, password hashing, credential validation, and JWT session token issuance.
Signup OTP Verification	PendingSignup model + SignupService	Pre-registration identity staging, email confirmation codes, rate-limiting.
Password Recovery & Tokens	AuthToken model + TokensService	Single-use, time-limited security token issuance, validation, and invalidation for password reset workflows.
Transactional Email Dispatch	EmailService	Asynchronous dispatch of verification codes, password reset links, and collaborator invitation notifications.
Project Workspaces	Project model + ProjectsService	Multi-tenant workspace partitioning, project ownership, metadata management, and cross-project data isolation.
Role-Based Access Control	ProjectMember model + MembersService + ProjectAccessService	Granular permission enforcement (admin, tester, viewer) guarding endpoints, test execution, and project settings.
Collaborator Invitations	ProjectInvitation model + InvitationsService	Secure tokenized email invitations, role pre-assignment, and acceptance/revocation lifecycle tracking.

Specification Ingestion	ApiSpecification model + SpecsService + SpecificationParser	OpenAPI 3.x parsing (JSON/YAML), circular reference resolution (prance), schema validation, and specification updates.
API Spec Generation	SpecGeneration model + SpecGenerationService + GenerationManager	Source code archive staging, OOPS LLM extraction orchestration, OpenAPI schema generation, and staging review/apply workflow.
API Endpoints & Schemas	Endpoint model + EndpointsService	Route extraction, parameter metadata indexing, manual endpoint injection, and route search/filtering.
Semantic Dependency Graph	DependencyGraph model + GraphService + OperationGraph	Directed graph generation mapping API operations to potential data-flow dependencies; supports Focus View topology.
Semantic Similarity Embedding	OperationDependencyComparator + EmbeddingModel	Cosine similarity calculation over property names and schemas using word embeddings to identify producer-consumer relations.
Test Run Orchestration	TestSuite model + TestSuitesService + JobManager	Test execution session management (time budgets, base URLs, route exclusions), engine process spawning, and status polling.
Multi-Agent RL (MARL)	AutoRestTest + QLearning (autoresttest-core)	Value-decomposition Q-learning loop, temporal-difference updates, agent coordination, and epsilon-greedy exploration decay.
LLM Contextual Value Generation	SmartValueGenerator + LanguageModel	Pre-generating realistic, context-aware parameter and body payload values via few-shot structured JSON prompts.
Captured HTTP Request Logging	TestCase model + RequestLog model	High-fidelity recording of every sent HTTP request (URL, method, headers, payload, response code, latency, response body).
Execution Replay	TestSuitesService (Replay Mode)	Deterministic sequential re-execution of previously captured HTTP requests to verify bug fixes without AI re-generation.

AI Request Descriptions	RequestDescriptionsService	Automated batch generation of plain-English summaries explaining the purpose and payload of each executed HTTP test case.
Test Analytics & Export	ReportsService	Aggregation of operation coverage, status-code distributions, error clustering, and report compilation into PDF and CSV formats.
Multi-Scope LLM Configuration	LlmSettings model + LlmSettingsService	Dynamic administrative management and runtime hot-reloading of LLM provider keys and models across 3 independent scopes.

4.2 Class-Responsibility-Collaborator (CRC) Cards

Table 4.2: AuthService CRC Card

AuthService	
Attributes	**Methods**
- userRepository: UserRepository - pendingSignupRepository: PendingSignupRepository - jwtService: JwtService - emailService: EmailService - bcryptRounds: number = 10	+ register(signupDto): Promise<PendingSignup> + verifySignupOtp(verifyDto): Promise<AuthPayload> + login(loginDto): Promise<AuthPayload> + validateUser(email, pass): Promise<User>
Responsibilities	**Collaborators**
Handles user registration, password hashing with bcrypt, credential validation, OTP email confirmation flow, and JWT session token issuance.	• User • PendingSignup • TokensService • EmailService

Table 4.3: TokenService CRC Card

TokensService	
Attributes	**Methods**
- authTokenRepository: AuthTokenRepository - jwtSecret: string - accessTokenExpiry: string = "15m" - refreshTokenExpiry: string = "7d"	+ generateAuthTokens(user): AuthTokens + createPasswordResetToken(email): Promise<string> + validatePasswordResetToken(token): Promise<User> + refreshSession(refreshToken): Promise<AuthTokens>
Responsibilities	**Collaborators**
Generates cryptographically signed JWT access and refresh tokens, manages single-use password reset tokens, and validates session expiration.	• AuthToken • User • EmailService

Table 4.4: ProjectService CRC Card

ProjectsService	
Attributes	**Methods**
- projectRepository: ProjectRepository - projectAccessService: ProjectAccessService - prismaService: PrismaService	+ createProject(userId, createDto): Promise<Project> + findAllForUser(userId, query): Promise<Project[]> + findOne(projectId, userId): Promise<Project> + deleteProject(projectId, userId): Promise<void>
Responsibilities	**Collaborators**
Manages multi-tenant project workspace creation, metadata updates, workspace data isolation, and owner-level cascade deletions.	• Project • ProjectAccessService • SpecsService • TestSuitesService

Table 4.5: ProjectAccessService CRC Card

ProjectAccessService	
Attributes	**Methods**
- memberRepository: MemberRepository - rbacPermissions: Map<Role, Permission[]>	+ checkAccess(userId, projectId, requiredRole): Promise<boolean> + getUserRole(userId, projectId): Promise<Role> + validateAdminOrOwner(userId, projectId): Promise<void>
Responsibilities	**Collaborators**
Enforces role-based access control (Admin, Tester, Viewer) across endpoints, guards project resources, and blocks unauthorized cross-tenant actions.	• ProjectMember • MembersService • InvitationsService • JwtAuthGuard

Table 4.6: SpecsService & SpecificationParser CRC Card

SpecsService & SpecificationParser	
Attributes	**Methods**
- specRepository: SpecRepository - pranceParser: PranceParser - endpointsService: EndpointsService	+ uploadSpecification(projectId, file): Promise<ApiSpecification> + parseAndValidate(fileContent): ParsedSpec + extractEndpoints(specId): Promise<Endpoint[]> + deleteSpecification(projectId): Promise<void>
Responsibilities	**Collaborators**
Parses and validates OpenAPI 3.0 specs (JSON/YAML), resolves circular $ref references, extracts endpoint paths/methods, and maintains schema versions.	• ApiSpecification • EndpointsService • GraphService • Prance Parser

Table 4.7: SpecGenerationService CRC Card

SpecGenerationService (OOPS Workflow)	
Attributes	**Methods**
- specGenRepository: SpecGenRepository - generationManager: GenerationManager - llmClient: OpenRouterClient - maxZipSizeMb: number = 50	+ uploadCodebaseZip(projectId, zipFile): Promise<SpecGeneration> + triggerOopsExtraction(genId): Promise<void> + getGenerationStatus(genId): Promise<JobStatus> + applyGeneratedSpec(genId): Promise<ApiSpecification>
Responsibilities	**Collaborators**
Stages codebase .zip archives, runs OOPS technology analysis and LLM endpoint method extraction, and manages developer schema review and application.	• SpecGeneration • GenerationManager • LLM Provider API • SpecsService

Table 4.8: TestSuiteService & JobManager CRC Card

TestSuitesService & JobManager	
Attributes	**Methods**
- suiteRepository: SuiteRepository - flaskEngineClient: FlaskEngineClient - activeJobs: Map<string, JobState>	+ createTestSuite(projectId, dto): Promise<TestSuite> + runTestSuite(suiteId, userId): Promise<JobResponse> + replayTestSuite(suiteId): Promise<TestSuite> + pollSuiteStatus(suiteId): Promise<SuiteMetrics>
Responsibilities	**Collaborators**
Configures test suite parameters (time budget, target URL, mutation rate), spawns asynchronous Flask testing jobs, tracks execution, and orchestrates replay.	• TestSuite • TestCase • Flask Engine Service • ReportsService

Table 4.9: MARL Engine CRC Card

MARLEngine (autoresttest-core)	
Attributes	**Methods**
- operationAgent: OperationAgent - parameterAgent: ParameterAgent - valueAgent: ValueAgent - dependencyAgent: DependencyAgent - qTableManager: QTableManager - mutationRate: float = 0.2	+ execute_test_loop(spec, time_budget): TestReport + select_action_joint(state): JointAction + update_q_tables_value_decomposition(reward, next_state): void + dispatch_request_with_mutation(request): Response
Responsibilities	**Collaborators**
Executes the multi-agent reinforcement learning loop, coordinates 4 RL agents via value decomposition Q-learning, mutates request payloads, and evaluates status codes.	• OperationAgent • DependencyAgent • SmartValueGenerator • Target REST API

Table 4.10: GraphService CRC Card

GraphService (SPDG)	
Attributes	**Methods**
- graphRepository: GraphRepository - embeddingModel: GloVeEmbeddingModel - similarityThreshold: float = 0.7	+ buildDependencyGraph(spec): DependencyGraph + computeCosineSimilarity(propA, propB): float + getProducerConsumerEdges(operationId): Edge[] + updateEdgeWeight(edgeId, status2xx): void
Responsibilities	**Collaborators**
Constructs the Semantic Property Dependency Graph (SPDG), calculates property name cosine similarity embeddings, identifies producer-consumer relationships, and updates weights.	• DependencyGraph • OperationGraph • EmbeddingModel • DependencyAgent

Table 4.11: ReportService CRC Card

ReportsService	
Attributes	**Methods**
- requestLogRepository: RequestLogRepository - llmReportingService: LlmReportingService - pdfExporter: PdfExporter - csvExporter: CsvExporter	+ generateSuiteReport(suiteId): Promise<ReportData> + getStatusCodeDistribution(suiteId): Promise<StatusMetrics> + explainFailureRootCause(logId): Promise<string> + exportReportPdf(suiteId): Promise<Buffer>
Responsibilities	**Collaborators**
Aggregates operation coverage and status code distributions, streams LLM failure explanations in plain English, and compiles PDF/CSV report exports.	• TestSuite • RequestLog • LLM Service • PDF/CSV Exporters

4.2 Persistent Data Sources

AutoRestTest utilizes distinct persistent data sources to store structured relational data, code archives, embeddings, caches, and raw execution logs. The table below outlines these data sources:

Table 4.12: Persistent Data Sources

Data Source	**Type**	**Description**
PostgreSQL	Relational DB	Primary data store that holds users, projects, specifications, endpoints, test suites, and captured request logs (14 models across Prisma ORM).
Specification & Code Store	File System	Staging area in uploads/ for incoming OpenAPI schemas (.yaml, .json) and codebase zip archives staged for AI specification generation.
Semantic Graph & Q-Table Cache	Binary Files (shelve)	cache/graphs/ and cache/q_tables/ – Content-addressed disk cache storing NLP embeddings, graph edges, and Q-tables keyed by spec_<sha256>.
Engine Process Logs	Log Files	jobs/<id>/stdout.log – Live stdout/stderr execution and error logs for active MARL testing runs.
Raw Engine Artifacts	JSON Files	data/<spec_name>/ – Detailed engine outputs including report.json, server_errors.json, and operation_status_codes.json.

Chapter 5

Interface Design

5.1 User Interface Architecture

The frontend uses a component-based architecture with NextJs and TanStack Query for context handling global state management. Below are the key UI objects and their hierarchical relationships:

Table 5.1: UI Objects and Actions

UI Object	**Type**	**Actions (Operations)**
GlobalNavbar	Navigation	Navigate routes, view notification badge, access profile settings
AuthForms	Form Group	Submit signup/login, verify OTP, recover/reset password
ProjectDashboard	Dashboard	View workspaces, initialize projects, track test metrics
SpecUploader	Dropzone Form	Upload raw OpenAPI schema (.yaml/.json), trigger parse validation
SpecGenerator	Multi-step Form	Upload source code zip, poll AI generation, apply generated schema
EndpointsList	Data Display	Filter active routes, manually add/delete endpoints, inspect parameters
DependencyGraph	Interactive Canvas	Render endpoint semantic property dependency graph, select active node
TestSuiteBuilder	Management	Configure suite parameters, trigger automated test execution
TestTimeline	Data Display	Monitor live execution progress, view real-time pass/fail metrics
LogViewer	Accordion	Expand HTTP records, view JSON request/response payloads
LiveReplayWidget	Controls	Execute live single-request replay from historical log data
AIExplanationModal	Floating Widget	Stream real-time failure root-cause analysis from LLM
ReportExporter	Controls	Download suite execution reports in PDF/CSV formats

RBACManager	Management	Send email invites, assign Admin/Tester roles, revoke workspace access
ProfileEditor	CRUD Form	Update user details, toggle notifications, manage account security
LLMSettings	Admin Control	Configure LLM API keys, Model name, RPM limit
ThemeToggle	Control	Switch interface between dark and light themes

5.2 UI Events & State Transitions

Table 5.2: Key UI Events and State Changes

User Event	Action	UI State Change
Click "Sign In"	POST /auth/login	Token stored -> Dashboard redirect
Submit OTP Code	POST /auth/verify-signup	Modal closes -> Workspace access granted
Create Project	POST /projects	Workspace initializes -> Redirect to Overview
Upload Source Zip	POST /projects/{id}/spec/generate	Polling state activates -> Generation tracks
Apply AI Spec	POST .../spec/generate/apply	Editor populates YAML -> Endpoints list updates
Upload Spec File	POST /projects/:id/spec	Editor mounts YAML, API extracts routes into EndpointsList
Add Manual Endpoint	POST /projects/{id}/endpoints	New route appends to EndpointsList
Click Graph Node	Local State Update	Detail panel slides in, related nodes highlighted
Click "Generate Suite"	POST /projects/{id}/test-suites	UI polls -> Test cases populate in data table
Trigger Test Run	POST .../test-suites/{id}/run	Badges turn 'Running', real-time timeline fills
Click "Live Replay"	POST .../request-logs/{id}/run	Request executes -> Inspector payload updates
Expand Request Log	GET .../test-suites/:id/test-cases	Row expands, HTTP headers and JSON payloads mount

Click "Request Explain"	POST .../request-logs/describe	Modal opens -> Plain-language analysis streams
Export Report	GET .../test-suites/{id}/report/export	apiDownload triggers -> PDF/CSV saved locally
Send Team Invite	POST /projects/{id}/invitations	Email dispatched -> 'Pending' row appends
Accept Invitation	POST /invitations/{token}/accept	Invite clears -> Project added to dashboard
Update LLM Config	PATCH /admin/llm-settings/{scope}	Global configuration syncs -> Toast appears

5.2 State Transition Diagram

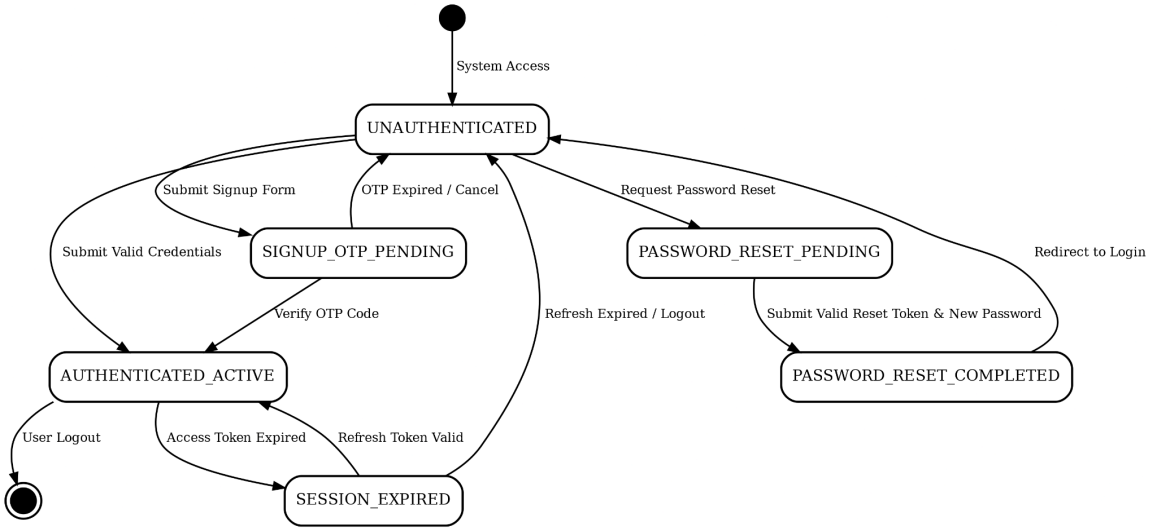


Figure 5.1: State Transition for Authentication and User Identity Lifecycle

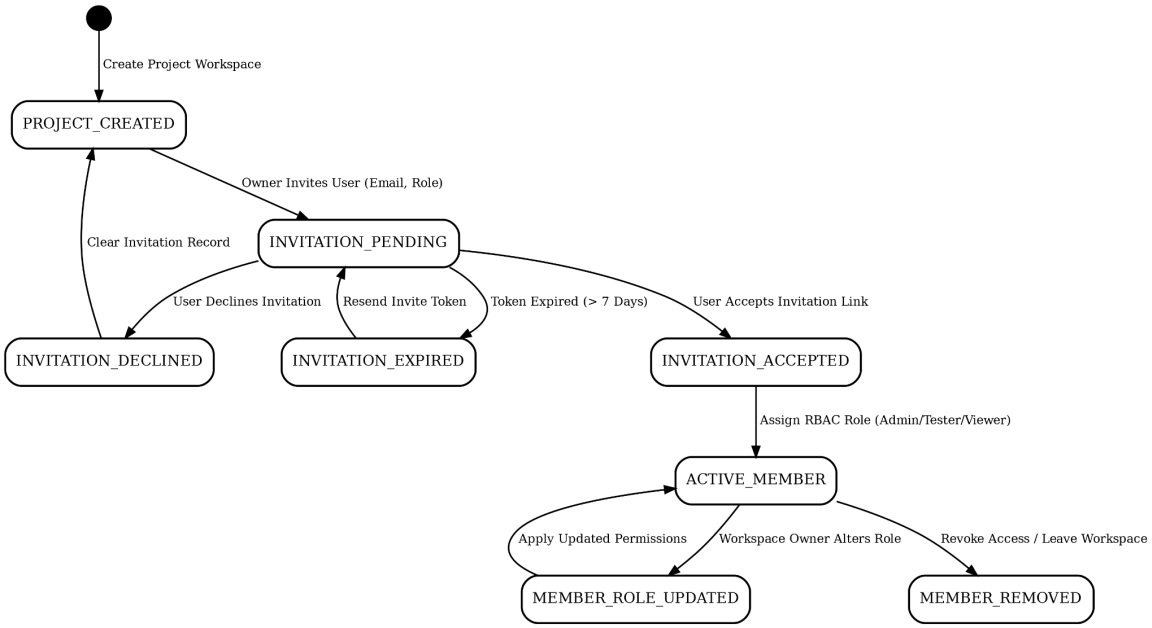


Figure 5.2: State Transition for Project Collaboration & Access Control

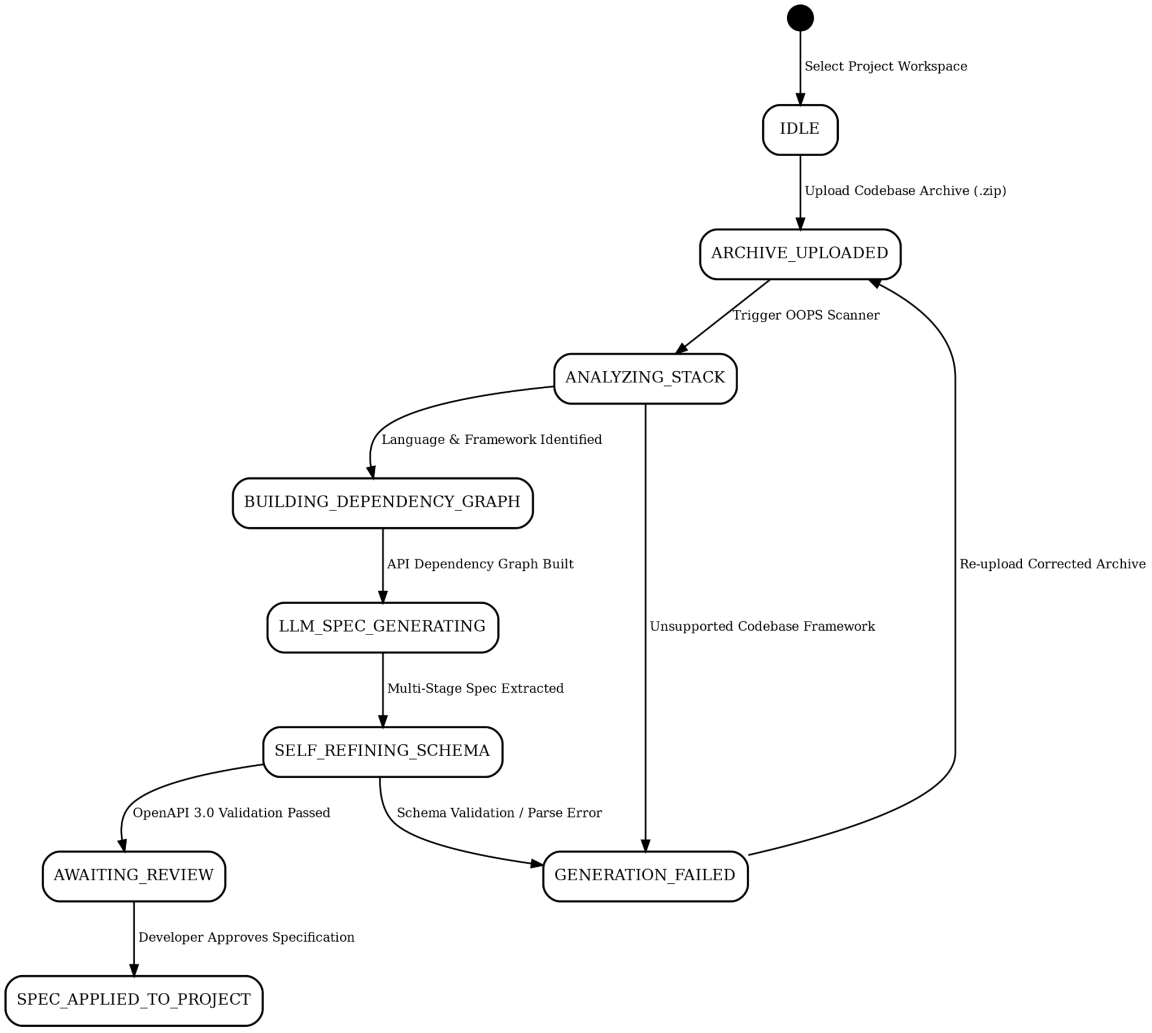


Figure 5.3: State Transition for Automated API Spec generation

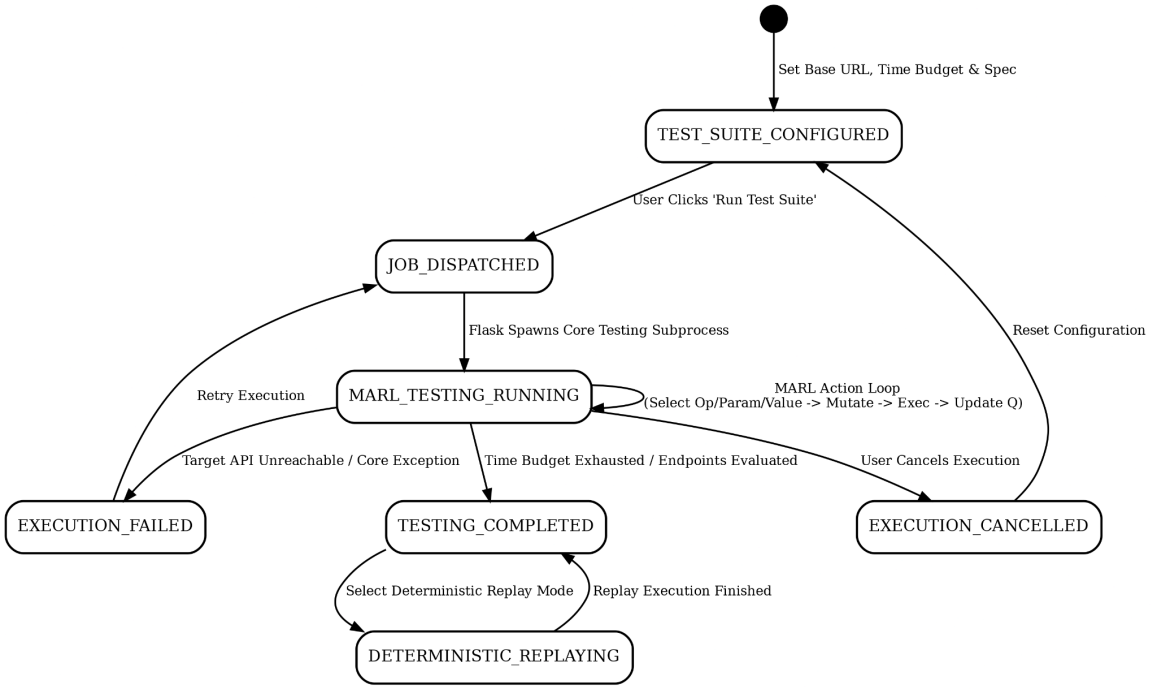


Figure 5.4: State Transition for Automated Test Suit Execution

Chapter 6
Testing

6.1 Testing Approach & Strategy

To ensure the reliability, data security and operational correctness of AutoRestTest, a comprehensive two-tier testing methodology was executed throughout the development lifecycle:

1. Unit Testing: Continuous, test-driven verification targeting the isolated logic of services, controllers, guards and algorithms across the application backend.
2. System & Black-Box Acceptance Testing: Scenario-driven end-to-end testing from the frontend for validating real-world workflows, permission boundaries and failure recovery.
3. Real World Evaluation: Some real world open source project from Github or publicly available REST API is testing using this AutoRestTest platform

6.2 Item Pass/Fail Criteria

To determine the success of an implemented feature, the following criteria are strictly applied:

- Pass Criteria: The module executes without throwing unhandled exceptions. Expected database states exactly match the actual database state. Unit tests pass with a 0 exit code.
- Fail Criteria: The application crashes or hangs indefinitely . Data corruption occurs. Unauthorized roles successfully access restricted endpoints. The UI fails to reflect the underlying state.

6.3 Risks and Contingencies

Table 6.1: Risks and Contingencies in testing

Risk Identified	**Impact**	**Contingency Plan**
LLM Non-Determinism	Automated tests relying on live LLM outputs may randomly fail due to varied responses.	Utilize ENGINE_MODE=mock during testing to inject static, predictable JSON responses in place of live LLM generation.
Large Graph Rendering	Cytoscape canvas may freeze the browser when rendering a massive enterprise API.	Implement strict node-limits and utilize the Focus View to only render a 3-column subset of active nodes at any given time.
Polling Timeouts	AI generation takes too long, causing the frontend to time out and display an error.	Implement background job tracking. The frontend polls for status without holding an open connection, timing out gracefully if the engine halts.

6.4 Unit Test Summary

Automated unit testing forms the backbone of the platform's verification, guaranteeing that regressions are caught immediately during development. The core backend rest api implementation is tested using Jest framework. A total of 21 test suites comprising 257 automated unit tests verify business logic, RBAC validations function correctly. External dependencies such as SMTP mail transport, the database layer, and external LLM endpoints are isolated via mock providers.

Table 6.2: Unit Test Summary

Test Suite / Service	Test Specification File	Test Cases	Scope & Logic Verified	Status
Authentication Service	auth.service.spec.ts	14	Login authentication, password verification, session issuance	PASS
Signup Service	signup.service.spec.ts	12	User registration validation, 6-digit confirmation code hashing	PASS
Token Service	tokens.service.spec.ts	9	JWT issuance, token refresh, cryptographically signed cookies	PASS
JWT Strategy & Guard	jwt.strategy.spec.ts	6	Bearer token extraction, authorization header validation	PASS
Invitations Service	invitations.service.spec.ts	16	Invite dispatch, 7-day expiration, token revocation, acceptance	PASS
Members Service	members.service.spec.ts	13	Role assignment (Admin, Tester, Viewer), permission revoking	PASS
Project Access Service	project-access.service.spec.ts	11	Multi-tenant isolation, ownership guard, unauthorized access blocks	PASS
Email Service	email.service.spec.ts	10	Resend integration, mock fallback mode, template formatting	PASS
Endpoint Extractor	endpoint-extractor.spec.ts	12	OpenAPI 3.0 path/method extraction, HTTP verb normalization	PASS
Endpoints Service	endpoints.service.spec.ts	15	Manual endpoint creation, schema persistence, safe deletion	PASS
Engine Service Client	engine.service.spec.ts	14	Microservice REST transport, health probes, retry & error logic	PASS
Graph Merge Algorithm	graph-merge.spec.ts	11	SPDG node/edge union, dependency resolution, weight updating	PASS

LLM Settings Service	llm-settings.service.spec.ts	8	Multi-provider configuration (NVIDIA, Gemini), key encryption	PASS
LLM Reporting Service	llm.service.spec.ts	10	Structured JSON prompt formatting, failure classification	PASS
Report Export Service	report-export.spec.ts	12	PDF rendering, CSV formatting, status code aggregation	PASS
Reports Service	reports.service.spec.ts	18	Coverage computation, error breakdowns, run analytics	PASS
Spec Generation Service	spec-generation.service.spec.ts	16	OOPS job dispatching, zip extraction, security validation	PASS
Specification Service	specs.service.spec.ts	14	OAS 3.0 YAML/JSON validation, versioning, replacement	PASS
Request Descriptions	request-descriptions.service.spec.ts	12	Natural-language explanation generator, batching logic	PASS
Test Suites Service	test-suites.service.spec.ts	24	Suite configuration, custom headers, run orchestration	PASS
Users Service	users.service.spec.ts	10	Profile updates, password alteration, account deletion	PASS
Total Backend Unit Tests	21 Suites	257	Complete backend domain, security, and data layers	100% PASS

6.5 Detailed Test Cases (Black box test)

In addition to automated unit tests, extensive system-level black-box testing was conducted against the fully deployed AutoRestTest platform. A total of 50 formal test cases were evaluated across seven core functional modules to validate complete user journeys, interface behavior, security barriers, and asynchronous run execution.

Table 6.3: Black Box test summary

Module	Module Name	Test Cases	Primary Scope	Status
Module 1	Authentication system	13	Registration, email verification, login, password recovery	PASS
Module 2	Project Management	3	Project creation, dashboard organization, safe deletion	PASS
Module 3	API Specification Management	12	Spec upload, validation, AI generation, dependency graph	PASS
Module 4	Test Suite Management	4	Run configurations, custom headers, endpoint exclusion	PASS
Module 5	Test Execution	6	One-click execution, live status, request inspection	PASS

Module 6	Test Results & Reports	5	Run reports, AI explanations, replay, history comparison	PASS
Module 7	Team Collaboration	7	Email invitations, token security, RBAC enforcement	PASS
Total	7 Functional Modules	50	End-to-end application lifecycle and security	100% PASS

Module 1: Authentication system

Test Case 1: User Registration with Valid Details

Test Scenario: A new user signs up for AutoRestTest for the first time.

Steps:

1. Open the registration page and enter a username, an email address, and a password of at least eight characters.
2. Submit the registration form.
3. Check the system for a newly created account.

Expected Outcome:

- The system accepts the request and tells the user that a six-digit confirmation code has been sent to their email address.
- No account has been created yet. The attempt is held aside as a pending signup, with the password already stored in a secure, unreadable form and the code stored only as a hash.
- No login session is issued at this stage, so an unconfirmed address gains no access to the platform.

Result: Pass.

Test Case 2: User Registration with Invalid or Missing Fields

Test Scenario: A user submits the registration form with information that does not meet the platform's rules.

Steps:

1. Submit the form with a two-character username.
2. Submit again with a username containing a hyphen.
3. Submit again with a five-character password.
4. Submit again leaving the email address blank.

Expected Outcome:

- Each attempt is rejected immediately with a clear, field-specific message: the username must be at least three characters, may contain only letters, numbers and underscores; the password must be at least eight characters; a valid email address is required.
- No pending signup is written for any of the rejected attempts.

Result: Pass.

Test Case 3: Registration with an Already Registered Email or Username

Test Scenario: A user tries to sign up using an email address or username that already belongs to a confirmed account.

Steps:

1. Complete registration and confirmation for the username "tester01".
2. Register again using the same email address.
3. Register once more using the same username but a different email address.

Expected Outcome:

- Both later attempts are rejected.
- The system says clearly which value is taken - "Email is already registered" or "Username is already taken".
- The existing account is unaffected.

Result: Pass.

Test Case 4: Re-registering an unconfirmed email address

Test Scenario: Someone registers with an email address they do not own, and later the real owner of that address signs up.

Steps:

1. Register with owner@example.com and never confirm the code.
2. Register again with owner@example.com using a different username and password.
3. Confirm using the code from the second email.

Expected Outcome:

- The second registration is accepted rather than rejected as a duplicate.
- It replaces the first pending signup, so only the second code works.
- The account is created for whoever could actually read the inbox.
- An unconfirmed address never permanently claims an email address on the platform.

Result: Pass.

Test Case 5: Email Confirmation with the Correct Code

Test Scenario: A newly registered user confirms their address with the code they received.

Steps:

1. Read the six-digit code from the confirmation email.
2. Enter the email address and the code on the confirmation screen.

Expected Outcome:

- The account is created and the pending signup is removed, both in a single step so a half-created account is not possible.
- A login token is returned together with the user's profile, in exactly the same shape as a normal login.
- The user lands on the dashboard already signed in.

Result: Pass.

Test Case 6: Email Confirmation with a Wrong Code

Test Scenario: A user mistypes the confirmation code repeatedly.

Steps:

1. Submit an incorrect six-digit code.

2. Repeat with different incorrect codes up to five times.
3. Try once more with the correct code.

Expected Outcome:

- Every wrong attempt is rejected and the number of remaining attempts is tracked.
- On the fifth wrong attempt the pending signup is destroyed and the user is told to register again.
- The correct code no longer works afterwards, and no account was ever created.

Result: Pass.

Test Case 7: Login with Wrong Password or Unknown Account

Test Scenario: Someone attempts to sign in with credentials that do not match.

Steps:

1. Enter a valid email address with the wrong password.
2. Enter an email address that has never registered.

Expected Outcome:

- Both attempts are denied.
- Both return exactly the same generic message, so the response does not reveal whether an account exists for that address.

Result: Pass.

Test Case 8: Password Reset

Test Scenario: A user forgets their password and resets it from the emailed link.

Steps:

1. Request a reset for a registered email address.
2. Open the emailed link and set a new password.
3. Try to sign in with the old password, then with the new one.
4. Try to use a session token that was issued before the reset.

Expected Outcome:

- A reset link is emailed and expires after thirty minutes; the secret is stored only as a hash, never in readable form.
- The new password works and the old one is refused.
- Every session created before the reset is rejected, so a stolen token cannot survive a password reset.
- The reset link cannot be used a second time.

Result: Pass.

Test Case 9: Password Reset for an Unknown Email

Test Scenario: Someone requests a reset for an address that has no account.

Steps:

1. Submit a reset request for an address that was never registered.

Expected Outcome:

- The response is the same neutral message shown for a registered address.

- The reset form cannot be used to discover which addresses have accounts on the platform.

Result: Pass.

Test Case 10: Update Display Name and Avatar Colour
Test Scenario: A signed-in user personalises their profile.

Steps:

1. Open Settings and change the display name.
2. Pick a different avatar colour and save.
3. Clear the display name and save again.

Expected Outcome:

- Both changes are saved and appear immediately in the header without a reload.
- Clearing the display name falls back to showing the username.

Result: Pass.

Test Case 11: Change Password
Test Scenario: A user changes their password from the settings page.

Steps:

1. Enter the current password and a new one, and save.
2. Try again with an incorrect current password.
3. Try setting the new password to the value it already has.
4. Use a session token that was issued before the change.

Expected Outcome:

- The valid change succeeds and a security notice is emailed to the account holder.
- A wrong current password is refused with "Current password is incorrect."
- Reusing the same password is refused.
- Sessions created before the change stop working.

Result: Pass.

Test Case 12: Notification Preferences
Test Scenario: A user turns off the two optional email notifications.

Steps:

1. Switch off "run finished" and "invitation" emails and save.
2. Complete a test run and have another user send an invitation.
3. Change the password from the settings page.

Expected Outcome:

- Preferences are saved and reflected on reload.
- Neither optional email is sent any more.
- Security notices and confirmation or reset emails are still delivered, because those cannot be switched off.

Result: Pass.

Test Case 13: Delete Account

Test Scenario: A user permanently closes their account.

Steps:

1. Press Delete account and submit without a password.
2. Submit with an incorrect password.
3. Submit with the correct password.
4. Try to sign in afterwards, and check a project the user had run tests in for someone else.

Expected Outcome:

- The first two attempts are refused: a valid session alone is not enough for an irreversible action.
- On the correct password, the account and everything it owns is removed.
- Signing in afterwards fails.
- Runs the user triggered inside other people's projects are preserved, with the attribution simply cleared.

Result: Pass.

Module 2: Project Management

Test Case 14: Create a Project

Test Scenario: A user creates a workspace to hold an API under test.

Steps:

1. Open the dashboard and press New project.
2. Enter a name and a description and create it.

Expected Outcome:

- The project is created with the creator as its owner and also recorded as a project administrator.
- It appears on the dashboard immediately, ready to receive a specification.

Result: Pass.

Test Case 15: Project Access Control

Test Scenario: An unrelated account tries to open someone else's project.

Steps:

1. Sign in as a user with no connection to a given project.
2. Open that project's page directly by its URL.
3. Also request a project id that does not exist at all.

Expected Outcome:

- Access to the existing project is denied because the caller is not the owner and not a member.
- A non-existent id is reported as not found.

- No project name, description, specification, or result data is revealed in either case.

Result: Pass.

Test Case 16: Delete a Project

Test Scenario: The owner permanently removes a project and everything under it.

Steps:

1. As the owner, press Delete and confirm with the account password.
2. Repeat as a project administrator who is not the owner.
3. Repeat as the owner but with the wrong password.
4. Press Delete repeatedly in a short period.

Expected Outcome:

- The owner's confirmed deletion removes the project together with its specification, endpoints, runs, captured requests, members and invitations.
- An administrator who is not the owner is refused, and so is a wrong password.
- Repeated attempts are throttled after five in fifteen minutes.

Result: Pass.

Module 3: API Specification Management

Test Case 17: Upload a Valid OpenAPI 3.0 Specification

Test Scenario: A user uploads the API contract that the platform will test against.

Steps:

1. Open the project's Specification tab and upload a valid OpenAPI 3.0 YAML file.
2. Repeat with the equivalent JSON file.
3. Open the Endpoints tab.

Expected Outcome:

- Both files are accepted and stored, and the API title and OpenAPI version are shown.
- Every path and method pair in the document is extracted and listed as an endpoint, with the operation summary used as its description.
- The endpoint count on the specification page matches what the Endpoints tab shows.

Result: Pass.

Test Case 18: Upload a Malformed Specification

Test Scenario: A user uploads a file that cannot be understood as an OpenAPI document.

Steps:

1. Upload a file with broken YAML indentation.
2. Upload a specification containing a reference that does not resolve.
3. Upload an empty file.

Expected Outcome:

- Each file is rejected with a specific message explaining what is wrong, rather than a generic error.
- The project's existing specification and endpoints are left exactly as they were.

Result: Pass.

Test Case 19: Replace an Existing Specification
Test Scenario: A user uploads a newer version of the API contract over an existing one.

Steps:

1. Upload a specification, add one endpoint manually, and build the dependency graph.
2. Upload a different specification over it.
3. Check the Endpoints and Dependencies tabs, and then an earlier completed run.

Expected Outcome:

- The new specification and its endpoints appear together, never a new specification with stale endpoints.
- Endpoints from the previous specification, including the manually added one, are cleared.
- The dependency graph built from the old specification is removed rather than shown as if it were current.
- Results of runs that have already been completed are untouched and still readable.

Result: Pass.

Test Case 20: Specification Permissions
Test Scenario: Members of different roles attempt to change the project's specification.

Steps:

1. As a viewer, attempt to upload and then to delete the specification.
2. As a tester, attempt the same.
3. As a project administrator and as the owner, attempt the same.
4. As a viewer, simply open the stored specification.

Expected Outcome:

- Only the owner and project administrators can upload or delete a specification.
- Testers and viewers are refused with a permission message.
- Viewers can still read the stored specification, so read access is not affected.

Result: Pass.

Test Case 21: Generate a Specification from a Source Archive
Test Scenario: A user has a REST API project but no OpenAPI document, and asks the platform to produce one from the source code.

Steps:

1. On the Specification tab choose "Generate from source".
2. Upload a .zip archive of the API's source code, optionally setting a title, version and extra directories to ignore.
3. Start the generation and watch the progress panel.

Expected Outcome:

- The job is accepted and queued, and the panel shows the pipeline advancing through its nine steps.
- The generation runs on its own queue, so a test run started at the same time is not blocked behind it.
- When it finishes, the generated OpenAPI document is presented for review rather than applied automatically.

Result: Pass.

Test Case 22: Reject an Invalid or Unsafe Source Upload

Test Scenario: A user uploads something the generator should not accept.

Steps:

1. Upload a .tar.gz archive instead of a .zip.
2. Upload a .zip larger than fifty megabytes.
3. Upload a .zip containing an entry with a "../" path, and another containing a symbolic link.

Expected Outcome:

- The non-zip archive is rejected with a message asking for a .zip.
- The oversized archive is rejected against the stated fifty-megabyte limit.
- The unsafe archives are refused during extraction, and nothing is written outside the job's own working directory.

Result: Pass.

Test Case 23: Review and Apply a Generated Specification

Test Scenario: A user reviews the generated document and adopts it as the project's specification.

Steps:

1. Inspect the generated document in the review panel.
2. Press Apply.
3. Open the Endpoints tab and the Specification tab.

Expected Outcome:

- The generated document is validated to exactly the same standard as an uploaded one before it is accepted.
- It becomes the project's specification, clearly marked as AI-generated.
- Its operations are extracted into endpoints, and the generation record is cleared.
- Nothing is replaced until Apply is pressed.

Result: Pass.

Test Case 24: Add an Endpoint Manually

Test Scenario: A user adds an operation that is not (or not yet) in the specification.

Steps:

1. Add POST /internal/health with a short description.

2. Open the run configuration form.

Expected Outcome:

- The endpoint is created, marked as manually added, and listed alongside the extracted ones.
- It is immediately available when configuring a test run.

Result: Pass.

Test Case 25: Delete an Endpoint

Test Scenario: A user removes an endpoint they no longer want tested.

Steps:

1. As the owner or a project administrator, delete a manually added endpoint.
2. As a tester, attempt the same.
3. Configure a new run afterwards.

Expected Outcome:

- The owner or administrator deletion succeeds and the row disappears from the list.
- The tester is refused with a permission message.
- The deleted endpoint no longer appears when configuring a run.

Result: Pass.

Test Case 26: Build the Project Dependency Graph

Test Scenario: A user asks the platform to work out how the API's operations depend on each other.

Steps:

1. Open the project's Dependencies tab and press Build.
2. Watch the status while the build runs.
3. Read the summary line once it completes.

Expected Outcome:

- The build is queued on the engine and the page tracks it until it is ready, without the user having to reload.
- On completion the view reports how many operations and how many resolved dependencies the graph holds.
- The build uses only the stored specification: no test run, no live API and no AI call is required.

Result: Pass.

Test Case 27: Inspect One Operation's Dependencies

Test Scenario: A user examines which operations feed into a given operation and which consume its output.

Steps:

1. Choose an operation from the list beside the diagram.
2. Read the diagram and the labels on the arrows.
3. Note the order in which the operations are listed and which one is selected first.

Expected Outcome:

- The diagram shows the operations that can supply this one's parameters on one side, the selected operation in the middle, and the operations that consume its response on the other.
- Each arrow is labelled with the parameter that connects the two operations.
- The list is ordered by how many connections each operation has, and opens on the most connected one, so the operations worth looking at are not buried alphabetically.

Result: Pass.

Test Case 28: Dependency Graph Attached to a Completed Run

Test Scenario: A user views the graph as it stood for a particular test run, with what the engine learned about it.

Steps:

1. Open a completed run and view its dependency graph.
2. Compare the weight shown on each dependency with the run's behaviour.
3. Open a run recorded before the graph feature was revised.

Expected Outcome:

- The run shows the graph that produced its results, even if the project's specification has since changed.
- Each dependency carries the weight the reinforcement-learning agent assigned to it during that run.
- An older run's stored graph is re-resolved when it is read, so old and new runs display in the same, readable form.

Result: Pass.

Module 4: Test Suit Management

Test Case 29: Configure a Test Run

Test Scenario: A user sets up a run of the AI testing engine against a live API.

Steps:

1. Press New run and give the run a name.
2. Enter the target base URL, a time budget in seconds and a mutation rate.
3. Save the configuration.

Expected Outcome:

- The run record is created in a pending state with exactly the settings entered.
- It appears at the top of the project's run list, ready to be started.

Result: Pass.

Test Case 30: Custom Authentication Headers

Test Scenario: A user configures a run against an API that requires authentication.

Steps:

1. Open the Advanced section of the run form.

2. Add an Authorization header carrying a Bearer token, and a second API-key header.
3. Save the run, then reopen it and inspect the run detail returned by the platform.
4. Start the run and inspect the captured requests.

Expected Outcome:

- The headers are stored and sent with every request the engine makes to the target API.
- The header values are never returned in any read response, so a credential entered once does not leak back out through the interface.

Result: Pass.

Test Case 31: Custom Header Validation

Test Scenario: A user enters header values that would be unsafe to forward.

Steps:

1. Add a header whose name contains a space.
2. Add a header whose value contains a line break.
3. Add twenty-one headers.

Expected Outcome:

- The invalid header name is rejected.
- The value containing a line break is rejected, so a header value cannot be used to inject extra headers into the outgoing request.
- The twenty-first header is rejected against the stated limit.

Result: Pass.

Test Case 32: Excluding Endpoints from a Run

Test Scenario: A user leaves out operations that are unsafe or pointless to fuzz.

Steps:

1. In the run form, mark two operations as excluded.
2. Start the run and let it complete.
3. Inspect the per-endpoint request summary.

Expected Outcome:

- The excluded operations are removed from the specification handed to the engine, so no request is generated for them.
- The remaining operations are tested normally.
- If an excluded endpoint has since been deleted from the project, the run still starts and simply ignores it.

Result: Pass.

Module 5: Test Execution

Test Case 33: One-Click Test Execution

Test Scenario: A user starts a configured run and follows its progress.

Steps:

1. Press Run on a configured run.
2. Stay on the page and watch the status without reloading.
3. Navigate to another page and come back.

Expected Outcome:

- The platform hands the specification and settings to the testing engine and returns straight away, without holding the browser.
- The status moves from pending to running to completed, and the counts of endpoints covered and test cases passed and failed appear as the run finishes.
- The interface stays usable throughout, and returning to the page shows the current state rather than a stale one.

Result: Pass.

Test Case 34: Running Without a Specification, or While Already Running

Test Scenario: A user starts a run in circumstances where it cannot proceed.

Steps:

1. Delete the project's specification and press Run on an existing run configuration.
2. On a running run, press Run a second time.

Expected Outcome:

- The first is refused with a message asking the user to upload a specification before running a test suite.
- The second is refused with "Test suite is already running", and the live run continues undisturbed.

Result: Pass.

Test Case 35: Re-running an Existing Run

Test Scenario: A user runs the same configuration again after fixing something in the target API.

Steps:

1. Open a completed run with results and captured requests.
2. Press Run again and let it finish.
3. Inspect the results and the captured request list.

Expected Outcome:

- The previous results, test cases and captured requests for that run record are cleared before the new execution begins.
- The displayed results always belong to the most recent execution, never a mixture of two.

Result: Pass.

Test Case 36: Per-Endpoint Request Summary

Test Scenario: A user reviews how many requests a run sent to each operation and how they turned out.

Steps:

1. Open a completed run's request list.

Expected Outcome:

- Each endpoint has one row showing how many requests were sent to it and how those split across the 2xx, 3xx, 4xx and 5xx response classes.
- Requests that did not match any known operation are collected in a separate bucket, which is itself a finding: it means the engine mutated the HTTP method.
- Matched endpoints are listed first, with the unmatched bucket last.

Result: Pass.

Test Case 37: Browse and Filter Captured Requests

Test Scenario: A user drills into one endpoint's requests and narrows them down.

Steps:

1. Open one endpoint's captured requests.
2. Page through the list.
3. Filter by one specific response code from the dropdown.
4. Copy the resulting URL and open it in a new tab.

Expected Outcome:

- Requests are listed in the order they were sent and are paginated so a run with thousands of requests still loads quickly.
- The response-code filter offers only the codes this run actually produced, and returns exactly the matching requests.
- The filtered view reopens unchanged from its URL, so a particular view can be shared with a teammate.

Result: Pass.

Test Case 38: Inspect a Single Request and Response

Test Scenario: A user examines exactly what was sent and what came back.

Steps:

1. Open one captured request from the list.

Expected Outcome:

- The full request is shown as it was sent: method, URL, headers and body.
- The full response is shown as it was received: status code, headers, body and how long it took.
- A body too large to store in full is clearly marked as truncated, rather than presented as if it were complete.

Result: Pass.

Module 6: Test Results & Reports

Test Case 39: Explain Requests

Test Scenario: A user asks the platform to describe, in plain language, what each generated request was testing.

Steps:

1. Open a completed run and press "Explain requests".
2. Wait for the pass to finish, watching the progress indicator.
3. Read the Description column in the request list.

Expected Outcome:

- Each request gains a single plain sentence describing what it tests, for example "Sends a title longer than the declared 50-character maximum".
- Progress is reported as the pass runs, and it continues automatically until every request has been described.
- Descriptions state the intent of the test, so the same request reads the same way whether it passed or failed.

Result: Pass.

Test Case 40: View a Run Report

Test Scenario: A user reviews the outcome of a completed run.

Steps:

1. Open the report for a completed run.

Expected Outcome:

- The overview shows how many endpoints were covered out of the total and as a percentage, the pass rate, the total, passed and failed test-case counts, and how long the run took.
- The distribution of response codes across the whole run is shown.
- Per-endpoint outcomes are listed, with a separate list of failures and a flag on any endpoint that produced server errors.

Result: Pass.

Test Case 41: Explain Failures

Test Scenario: A user asks for plain-language explanations of the run's failures.

Steps:

1. Press "Explain failures" on a run with failures.
2. Wait for the explanations to appear.
3. Press it a second time.

Expected Outcome:

- Each failed endpoint gains a short explanation of the likely cause, written in plain language.
- The second press reuses explanations already produced and only fills in the ones still missing, so an interrupted first attempt can simply be repeated.

Result: Pass.

Test Case 42: Replay a Completed Run

Test Scenario: A user re-sends a recorded run's exact request sequence to check whether the API still behaves the same way.

Steps:

1. Press Replay on a completed run.
2. Watch the replay run to completion.
3. Return to the original run.

Expected Outcome:

- A new run is created, marked as a replay, and the original run's captured requests are re-sent in their recorded order against the target.
- No AI generation is involved, so the replay is a deterministic regression check rather than a fresh set of tests.
- The original run's results are never overwritten, and both runs remain available side by side.

Result: Pass.

Test Case 43: Run History and Comparison

Test Scenario: A user compares a run against its replays to spot a regression over time.

Steps:

1. Open the history panel for a run that has several replays.
2. Read the pass and fail counts along the timeline.

Expected Outcome:

- The original run and every replay of it are listed oldest first, each with its outcome counts.
- A change in behaviour between two points in time is visible at a glance, because every entry sends the same fixed request sequence.

Result: Pass.

Module 7: Team Collaboration

Test Case 44: Invite a Collaborator

Test Scenario: A project administrator brings a teammate into the project.

Steps:

1. Open the Team tab and invite an email address with the tester role.
2. Check the invitation list and the invited person's inbox.

Expected Outcome:

- A pending invitation is created with a unique token, valid for seven days.
- An invitation email is sent to the address, and the invitation is listed as pending on the Team tab.

Result: Pass.

Test Case 45: Accept an Invitation

Test Scenario: An invited user joins the project.

Steps:

1. Sign in as the invited user and open the Invitations page.
2. Accept the invitation.
3. Open the project.

Expected Outcome:

- The user becomes a member with exactly the role they were invited with, no more.
- The invitation is marked accepted, and the project appears in the user's project list.

Result: Pass.

Test Case 46: Invitation Security and Expiry

Test Scenario: Someone tries to use an invitation that is not theirs, or is no longer valid.

Steps:

1. While signed in as one account, attempt to accept an invitation addressed to a different email address.
2. Attempt to accept an invitation that has passed its seven-day expiry.
3. Attempt to accept an invitation that has already been accepted or declined.

Expected Outcome:

- The wrong recipient is refused, so an invitation link alone is not enough to join a project.
- ·An expired invitation is refused with a message saying so.
- An invitation that has already been resolved cannot be used again.

Result: Pass.

Test Case 47: Resend, Revoke and Decline an Invitation

Test Scenario: An administrator manages outstanding invitations, and an invitee turns one down.

Steps:

1. Resend a pending invitation.
2. Revoke a different pending invitation and try its link afterwards.
3. As an invitee, decline a third invitation.

Expected Outcome:

- Resending re-sends the email for the same invitation, and reports clearly if the email could not be sent.
- A revoked invitation is removed and its link no longer works.
- A declined invitation is marked declined and creates no membership.

Result: Pass.

Test Case 48: Role Enforcement: Viewer

Test Scenario: A viewer confirms they can read everything and change nothing.

Steps:

1. As a viewer, attempt to upload a specification, add an endpoint, configure a run, start a run, replay a run, build a dependency graph, re-run a single captured request, and invite a member.
2. As the same viewer, open the specification, endpoints, dependency graph, run results, captured requests and reports.

Expected Outcome:

- Every attempt to change something is refused with a permission message.
- Every read succeeds, so a viewer is a genuinely useful read-only role rather than a locked-out one.

Result: Pass.

Test Case 49: Role Enforcement: Tester and Administrator

Test Scenario: A tester and an administrator confirm the boundary between their roles.

Steps:

1. As a tester, configure and start a run, replay it, re-run a single captured request, build the dependency graph and request explanations.
2. As the same tester, attempt to upload a specification, delete a run, delete an endpoint and manage members.
3. Repeat both groups as a project administrator.
4. As a project administrator, attempt to delete the project.

Expected Outcome:

- The tester can do everything in the first group and is refused for everything in the second.
- The administrator can do both groups.
- Deleting the project remains with the owner alone, even for an administrator.

Result: Pass.

Test Case 50: Manage Members

Test Scenario: An administrator adjusts the team, and a member leaves.

Steps:

1. Change a member's role from viewer to tester.
2. Remove a member from the project.
3. As a member, leave the project.
4. As the owner, attempt to leave.

Expected Outcome:

- The role change takes effect on the member's very next action.
- A removed member loses access to the project immediately.
- A member can leave on their own initiative.

The owner is told to delete the project instead of leaving it, so a project is never left without an owner.

Result: Pass.

Chapter 7
Real-World Evaluation

7.1 Overview

To evaluate the practical effectiveness of the AutoRestTest platform, an empirical evaluation was conducted against 15 real-world REST API projects. The primary objective of this evaluation was to validate the platform's ability to autonomously parse OpenAPI specifications, resolve complex data dependencies using the Semantic Property Dependency Graph (SPDG), and leverage the Multi-Agent Reinforcement Learning (MARL) and LLM-driven engine to maximize endpoint coverage and discover genuine server-side faults (5xx errors) without requiring manual test case creation.

7.1 Evaluation Methodology

For each of the 15 APIs evaluated, a systematic testing procedure was executed utilizing the AutoRestTest platform:

1. Target Environment Setup: If the target project provided a live hosted platform, it was tested directly. If no live hosted platform was available, the project was self-hosted locally using Docker to ensure a stable testing environment.
2. API Specification Management: The official OpenAPI Specification (OAS) file provided in the target project's source code was used for the testing.
3. Execution Configuration: Test suites were configured with a time budget based on number of endpoints in the target API, generally ranging from 10 to 30 minutes.
4. LLM Usage and Generation: The testing engine utilized the free LLM model provided by NVIDIA (model: nvidia/nemotron-3-super-120b-a12b), operating within a rate limit of 35 Requests Per Minute (RPM).
5. Analysis: The system's generated reports were analyzed for total endpoints covered, HTTP status code distributions (2xx, 4xx, 5xx), and AI-generated failure explanations.

7.3 Evaluation Summary

Table 7.1: Real-World Evaluation Summary

Project Name	Source	Total Endpoints	Time Budget (sec)	Endpoint Coverage (%)	Request Sent	Successful (2xx)	Server Error (5xx)	Faulty Endpoints
HomeLogger	github.com/Francis Laboratories/homelogger	37	1200	92%	2208	1134	94	4
ApogeoAPI	api.apogeoapi.com/api/docs	46	1200	93%	496	249	10	3

UK Parliament Bills API	bills-api.parliament.uk	22	1500	77%	4912	1857	2	2
UK Parliament Members API	members-api.parliament.uk	43	1200	91%	1306	590	35	1
Snipe-IT	github.com/grokability/snipe-it	145	1500	98%	3162	2104	6	5
RealWorld	github.com/realworld-apps/realworld	17	1200	88%	949	230	0	0
Holiday	github.com/nager/Nager.Date	6	300	100%	611	361	51	1
LubeLogger	github.com/hargata/lubelog	76	1500	62%	1016	314	30	19
Grocy	https://grocy.info	44	1200	82%	1198	414	152	20
IUCN Red List	api.iucnredlist.org	50	900	88%	894	423	3	3
CosmyDay	cosmyday.com/api-docs	17	1200	94%	414	249	0	0
VATcomply	github.com/madisvain/vatcomply	7	300	100%	468	325	0	0
CareerStory	github.com/CodeWithIsmail/CareerStory-backend	18	600	72%	680	170	3	1
Laravel Blog	github.com/guillaumebriday/laravel-blog	23	1500	91%	2250	1235	10	4
CycleCalcs	www.cyclecalcs.com/api.html	30	900	100%	1130	501	0	0

Chapter 8
User Manual

8.1 Introduction

AutoRestTest is a web-based, AI-powered, collaborative platform for automated REST API testing. It brings together several capabilities that are normally spread across separate tools into a single project workspace:

- **API specification management:** Upload an existing OAS (OpenAPI Specification) document in .json or .yaml format, or generate one automatically by uploading a zip of the API's source code.
- **Dependency visualization:** a semantic dependency graph shows which API operations rely on data produced by other operations, before a single request is ever sent.
- **AI-driven test execution:** a multi-agent reinforcement-learning and LLM-backed engine generates and runs a test suite against a live target API, with real-time status and a detailed report.
- **Team collaboration:** invite teammates to a project with a specific role (Admin, Tester, or Viewer) and work together on the same specification, endpoints, and test runs.

This manual walks through every page and feature of the application, in the order a new user would typically encounter them, so that any user regardless of technical background can get started quickly.

8.2 Installation

AutoRestTest can be accessed as a publicly available cloud application or self-hosted locally on your own infrastructure.

8.2.1 Publicly Available Application

The public version of AutoRestTest is fully web-based and requires no installation on your computer. The entire platform runs directly in your browser.

URL: **https://autoresttest.vercel.app**

8.2.2 Self Hosting

If you prefer to run AutoRestTest locally or need to configure a custom LLM provider, you can self-host the platform. The repository is a monorepo containing four independent subprojects, meaning there is no root-level package manager. You can deploy using Docker or configure each service manually.

Github Repository: **https://github.com/CodeWithIsmail/AutoRestTest**

Method A: Docker (Recommended)

1. Clone Repository:

```
git clone https://github.com/CodeWithIsmail/AutoRestTest.git
cd AutoRestTest
```

2. Configure environment variables:
   ```
   cp docker-compose.env.example .env
   ```
3. Run the application:
   ```
   docker-compose -f docker-compose.prod.yml up -d
   ```

Method B: Manual Setup

To run the platform manually without Docker, you must configure and start the four subprojects individually.

1. Clone the repository and configure environment variables:
   ```
   git clone https://github.com/CodeWithIsmail/AutoRestTest.git
   cd AutoRestTest
   ```
   Copy the .env.example files to .env in all four subdirectories (backend, engine-service, frontend, and autoresttest-core) and fill in your specific values.
2. One-Time Dependency Installation:
   - **Python Engine** (autoresttest-core):
     ```
     cd autoresttest-core
     poetry install
     ```
   - **Flask Wrapper** (engine-service):
     ```
     cd ../engine-service
     python -m venv .venv
     .\.venv\Scripts\Activate.ps1
     pip install -r requirements.txt
     ```
   - **NestJS Backend** (backend):
     ```
     cd ../backend
     npm install
     npx prisma generate
     npx prisma migrate deploy
     ```
   - **Next.js Frontend** (frontend):
     ```
     cd ../frontend
     npm install
     ```
3. Running the Services:

   Start the services in the following order, keeping a dedicated terminal open for each:
   - **Terminal 1**
     ```
     cd engine-service
     .\.venv\Scripts\Activate.ps1
     python wsgi.py
     ```
   - **Terminal 2**
     ```
     cd backend
     npm run start:dev
     ```
   - **Terminal 3**
     ```
     cd frontend
     npm run dev
     ```

8.3 Getting Started

Open your browser and navigate to AutoRestTest URL (https://autoresttest.vercel.app). If you are not already signed in, the application takes you to the Landing page.

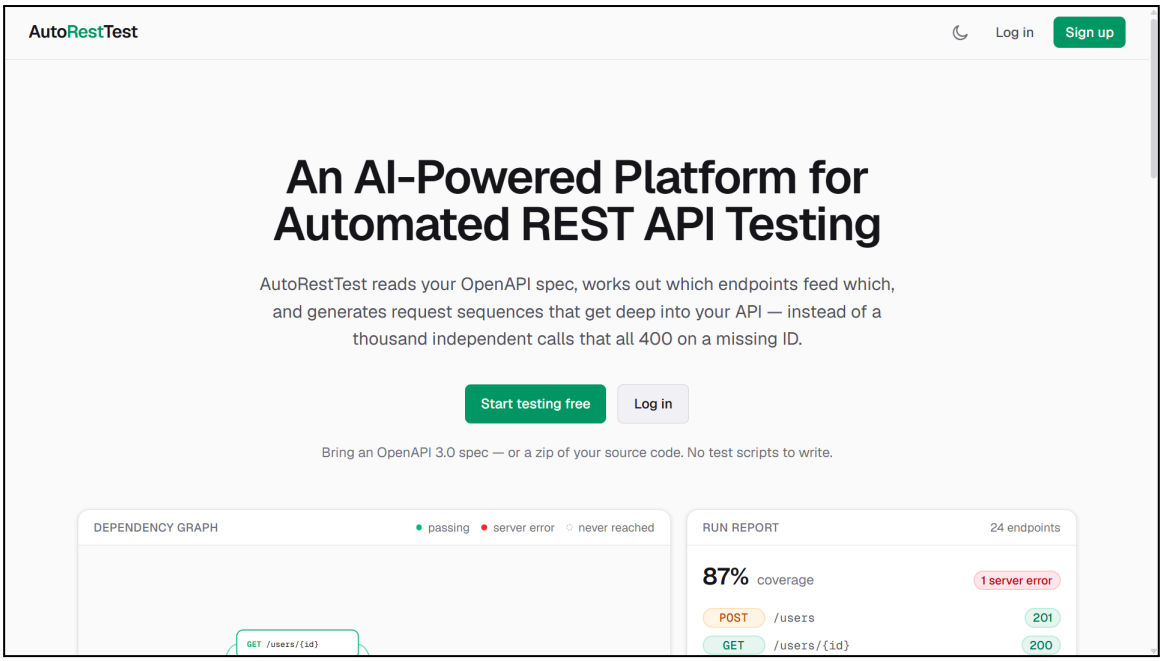


Figure 8.1: AutoRestTest Landing Page

8.4 Account Creation

8.4.1 Register

If you do not already have an account, you can create one from the Register page.

1. On the Landing page, click "Sign Up" on the right side of the top navigation row.
2. Fill in the registration form:
   - **Username:** a unique name, 3–32 characters. This becomes your permanent sign-in name and cannot be changed later.
   - **Email:** a valid email address you have access to. This also becomes your permanent email address.
   - **Password:** at least 8 characters.
3. Click "Create account".

Registration does not create your account immediately, it only reserves it. AutoRestTest sends a six-digit verification code to the email address you entered, and you are taken to the Confirm Your Email screen.

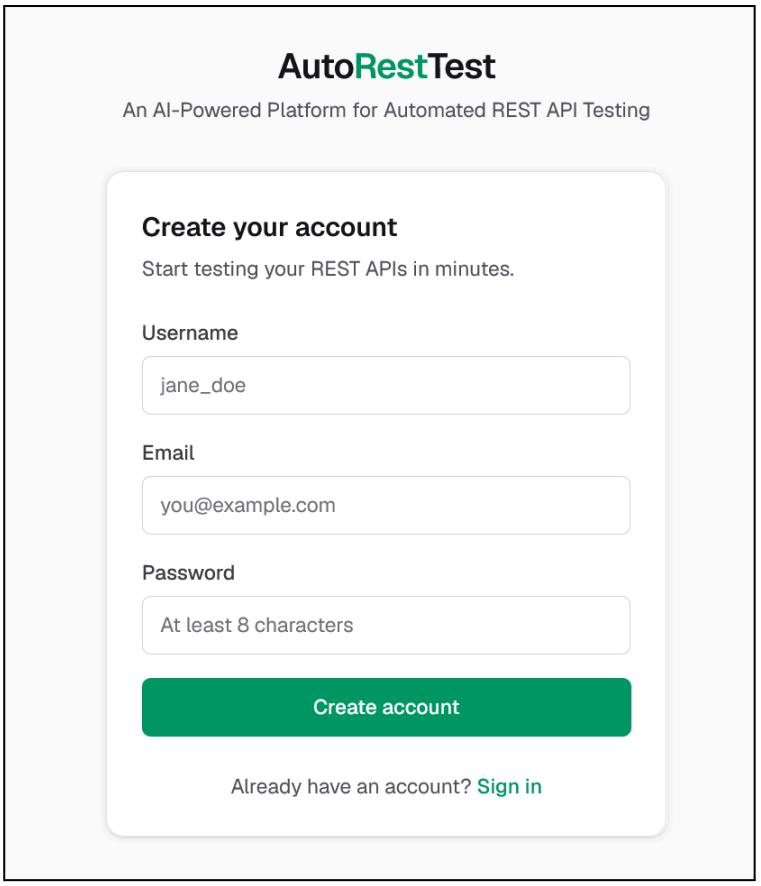


Figure 8.2: Account register page

8.4.2 Email Verification

The Confirm Your Email screen shows six input boxes for the six-digit code that was emailed to you. Typing (or pasting) the code fills the boxes automatically and, once all six digits are entered, the form submits itself, there is no separate button to click. Your account is only created at this step, once the code is verified.

- If you did not receive the code, use the "Resend the code" link. Once used, it is disabled for 60 seconds before it can be used again.
- If you entered the wrong email address, use the "Sign up again" link to restart registration.

On success you are signed in automatically and taken straight to your Projects page. There is no separate account-approval step and no unverified state to wait out.

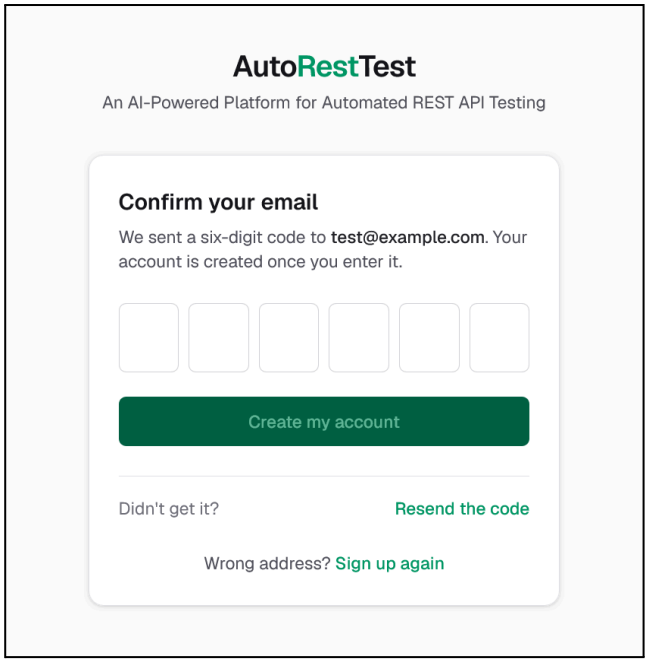


Figure 8.3: Email verification page

8.5 Sign In

1. On the Login page, enter your Email or username and your Password in a single combined field for the identifier. Either your email address or your username is accepted.
2. Click "Sign in".
3. If your credentials are correct, you are taken directly into the application, on your Projects page.

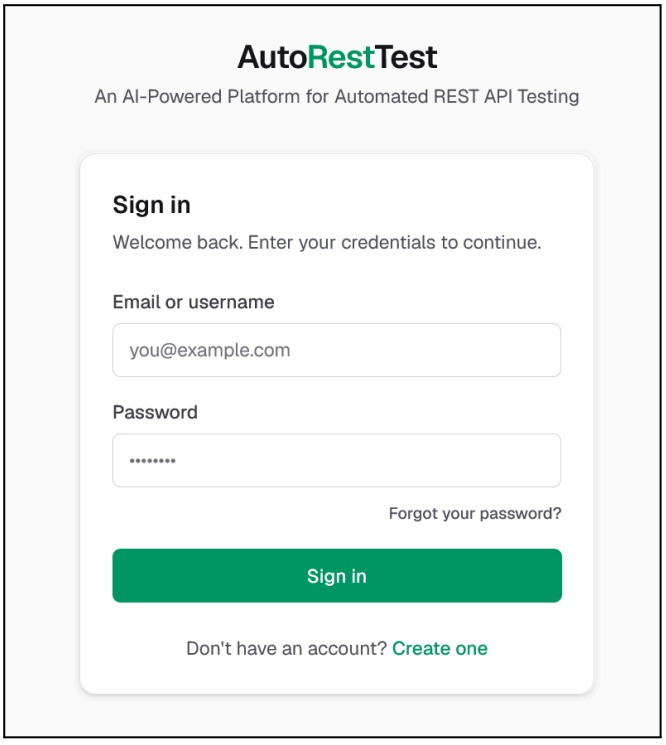


Figure 8.4: Login page

8.5.1 Password Reset

If you have forgotten your password, click "Forgot your password?" on the Login page.

1. Enter the email address on your account and click "Send reset link".
2. Check your inbox for a reset link. For security, the screen shows the same confirmation message whether or not the address is registered, so it cannot be used to discover which emails have an account.
3. The link works once and expires after 30 minutes.
4. Opening the link takes you to the Choose a New Password screen, where you enter and confirm a new password (at least 8 characters, and the two entries must match).

Resetting your password signs you out of every device and browser you were previously signed in on, including the one you are using now. Sign in again with your new password.

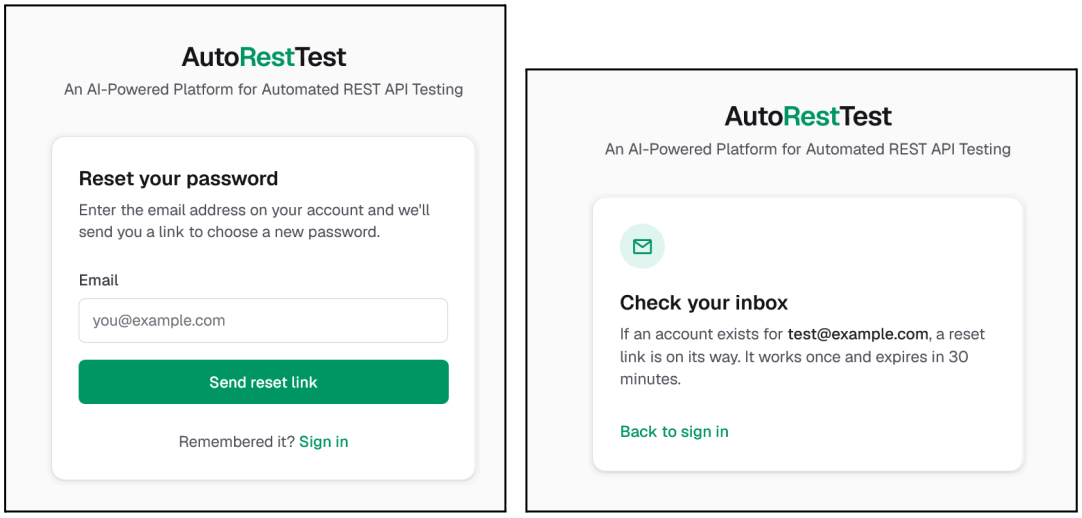


Figure 8.5: Reset password page

8.6 Application Layout

After signing in, the application uses a single top bar for all navigation — there is no sidebar. From left to right, the top bar contains:

- **AutoRestTest logo,** which always returns to your Projects list.
- **Project switcher,** visible whenever you are inside a project; shows the current project's name and lets you jump directly to any other project without returning to the list first.
- **Invitations bell icon,** shown only when you have at least one pending project invitation, with a badge showing how many.
- Your display name, and a light/dark theme toggle.
- **Account menu,** opened from your avatar, with links to "Invitations", "Settings", and "Sign out".



Figure 8.6: Navbar

8.7 Projects

The Projects page is what you see immediately after signing in. It is the application's home page. Each project is an independent workspace holding one API specification, its endpoints, its dependency graph, its test runs, and its team.

8.7.1 Viewing Your Projects

The Projects page offers:

- **A search box,** which filters by project name or description.
- **A role filter,** with options All roles, Owner, Admin, Tester, and Viewer.
- **A view toggle,** switching between a Table view and a Cards view; your choice is remembered for next time.

The table view's columns such as Name, Members, Role, and Last activity can each be clicked to sort the list; clicking the same column again reverses the sort order. A Status column shows badges for the project's specification state (No spec / Spec uploaded / Spec generated), any AI generation in progress (Generating… / Awaiting review / Generation failed), and the status of its most recent test run.

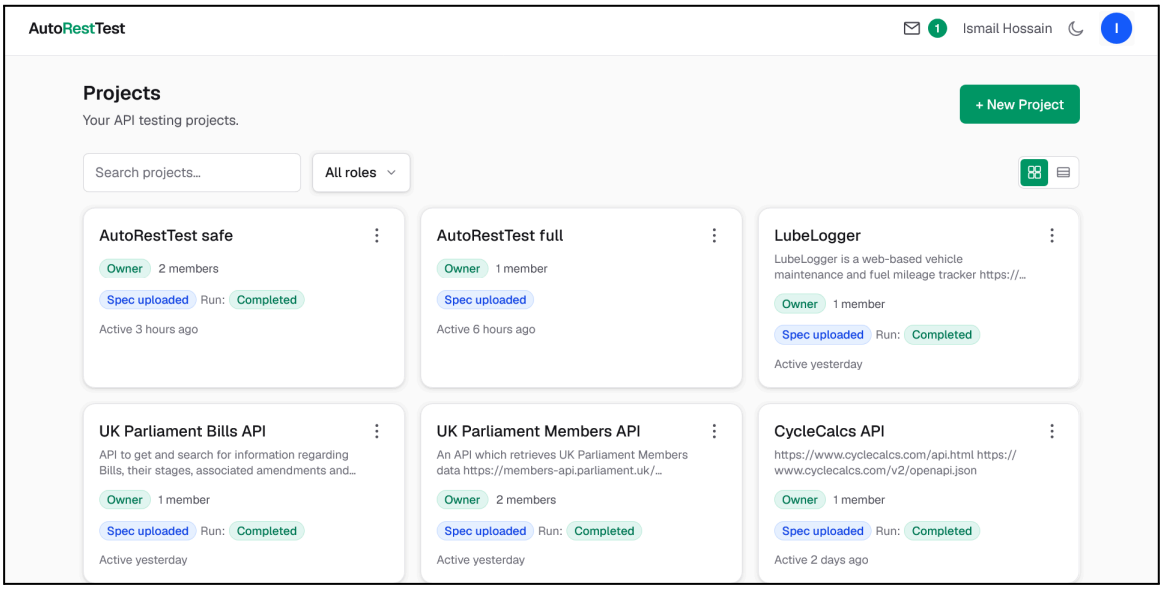


Figure 8.7: Homepage dashboard page

8.7.2 Project Creation

1. Click "+ New Project".
2. Enter a Name (required) and, optionally, a Description.
3. Click "Create project".

You are taken directly into the new project, on its Overview tab.

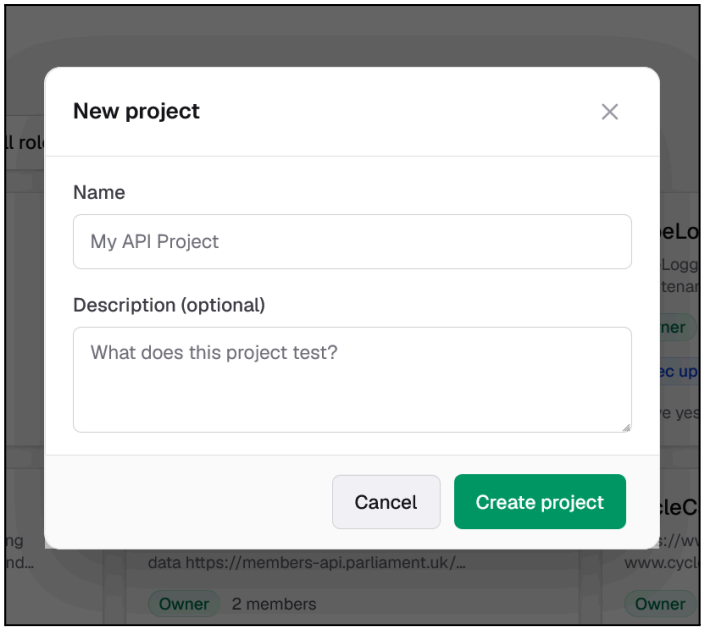


Figure 8.8: Create new project form

8.7.3 Editing or Deleting a Project

Only a project's owner can edit or delete it. From the project's own page, use the Edit and Delete buttons next to the project name; from the Projects list, open the row menu on a project you own. Deleting a project asks for confirmation and permanently removes the project and everything in it such as its specification, endpoints, dependency graph, test runs, and captured requests.

8.8 Project Roles and Permissions

Every project has one owner and any number of members, each holding one of three roles. Permissions are enforced both in the interface and on the server.

Role	Can do
Owner	Everything below, plus rename or delete the project itself, and manage all members.
Admin	Manage the API specification and endpoints; configure, run, and manage test suites; manage team members and invitations.
Tester	Configure and trigger test runs; run individual captured or live requests. Cannot change the specification, endpoints, or team.
Viewer	Read-only access to everything in the project; cannot change or run anything.

8.9 Project Overview

Opening a project lands on its Overview tab. A tab row beneath the project name shows Overview, API Spec, Endpoints, Dependencies, Test Suites, Team; switches between the project's sections; it stays visible throughout.

The Overview tab shows the project's description, its owner, member count, creation date, and last-updated date, alongside a short list of its team (owner and members, each with a role badge).

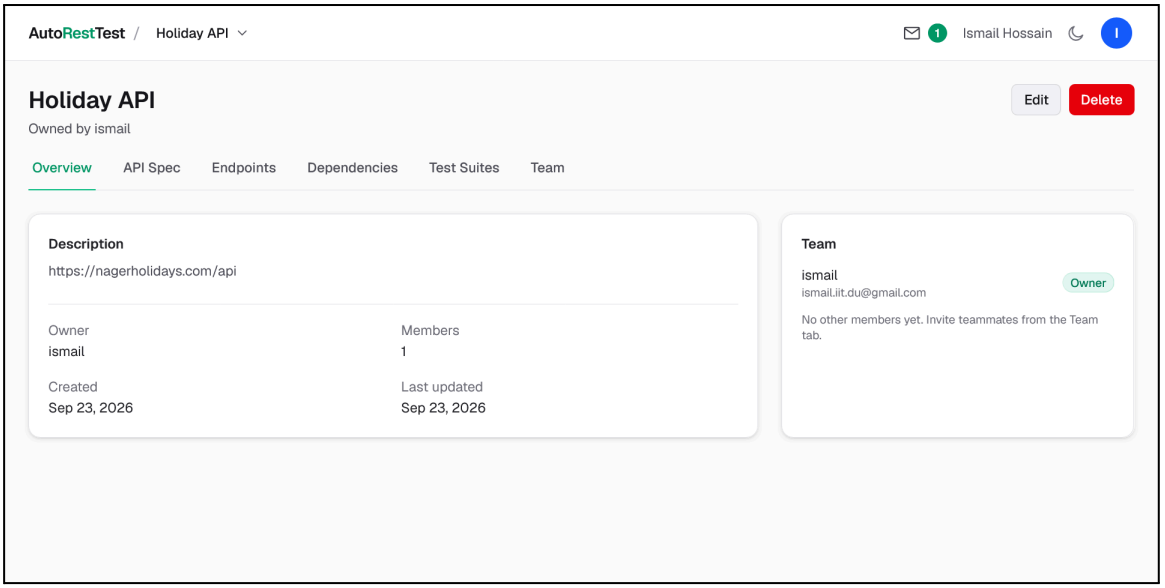


Figure 8.9: Project overview

8.10 API Specification

The API Spec tab is where a project's OpenAPI document lives. Project owners and admins are offered two ways to provide one, selected with a toggle at the top of the page: Upload OAS file, or Generate from source code. Testers and viewers see only the specification that is currently in place.

8.10.1 Uploading an OpenAPI File

1. Select "Upload OAS file".
2. Drag an OpenAPI document onto the drop zone, or click it to browse for a file. Accepted formats are .json, .yaml, and .yml, describing an OpenAPI 3.x document.

Endpoints are extracted automatically as soon as the file is accepted. If the project already has a specification, uploading a new one asks for confirmation first, since replacing it re-extracts the endpoint list and removes any endpoints that were added manually. Once a specification is in place, its card shows the document's title, its OpenAPI version, an AI-generated badge if it came from the code-generation flow described below, its endpoint count, and its upload date along with Replace and Delete buttons. The raw specification text is shown below, with buttons to copy it or download it as a file.

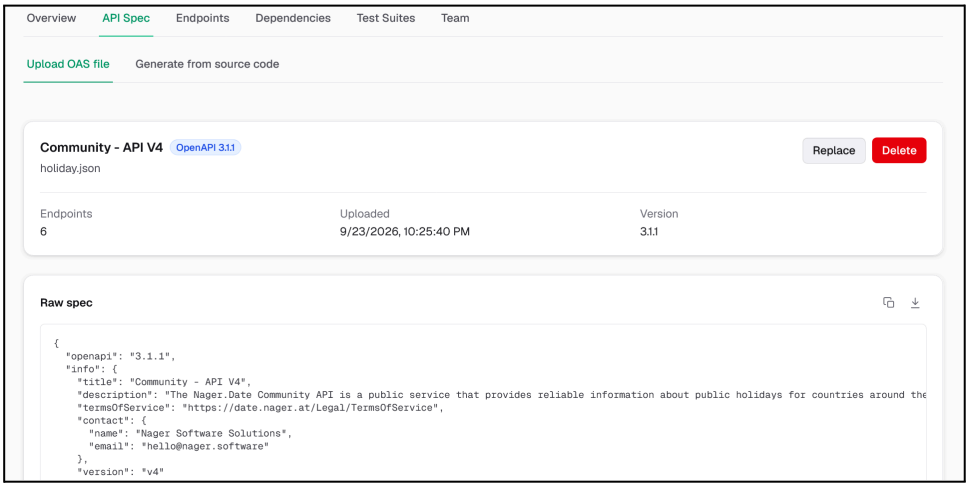


Figure 8.10: API Specification management

8.10.2 Generating a Specification from Source Code

If a project does not yet have an OpenAPI specification, AutoRestTest can generate one automatically by reading the API's own source code with an AI pipeline.

1. Select "Generate from source code".
2. Fill in the API title and Version (both prefilled from the project's own name and "1.0.0"), and, optionally, adjust Exclude directories by a comma-separated list of folders to skip (vendored dependencies and build output are pre-filled, since excluding them makes the analysis considerably faster and cheaper).
3. Drag the project's source code onto the drop zone as a single .zip archive (up to 50 MB), or click to browse for one.

Generation runs in the background and can take a long time for a large codebase. The page shows a progress bar, the current step, and an elapsed-time counter, and updates automatically, so it is safe to leave the page and come back later. A "Cancel" button discards a generation that is still running.

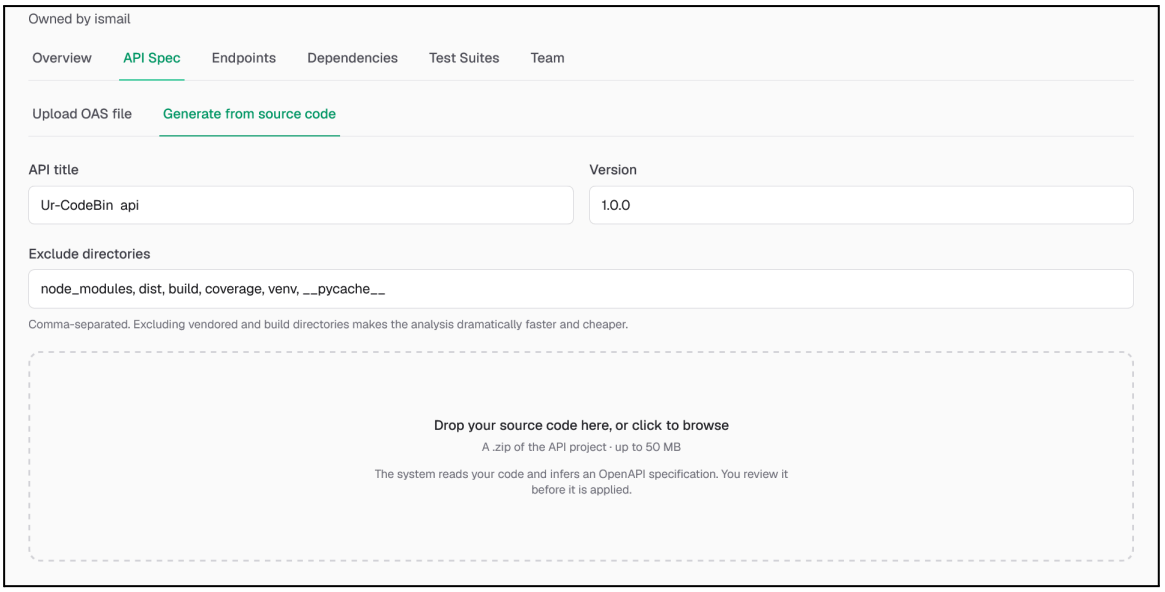


Figure 8.11: Auto generate API Specification

Once generation finishes, the specification is shown for review before it changes anything in the project. Any warnings from the pipeline; for example, a note that a required tool was unavailable and a lower-fidelity conversion was used are shown alongside it. Clicking "Review generated spec" expands the full generated document. From here you can either "Discard" the result, or click "Use this specification" to apply it as the project's specification (if one already existed, this asks for confirmation first, for the same reason as replacing an uploaded file).

8.11 API Endpoints

The Endpoints tab lists every operation extracted from the project's specification, plus any added manually. A search box filters by path, method, or description.

Each row shows:

- An HTTP method badge (GET, POST, PUT, PATCH, DELETE, …).
- The endpoint's path, with inline indicators for whether authentication is required, how many parameters it takes, and whether it accepts a request body.

- Its description.
- A Source badge "From spec" for endpoints extracted from the uploaded or generated specification, or "Manual" for ones added by hand.

Clicking a row expands its full detail directly beneath it: authentication requirements, every parameter (with where it is sent, path, query, header, or cookie, its type, and whether it is required), the request body schema with an example payload, and the documented responses.

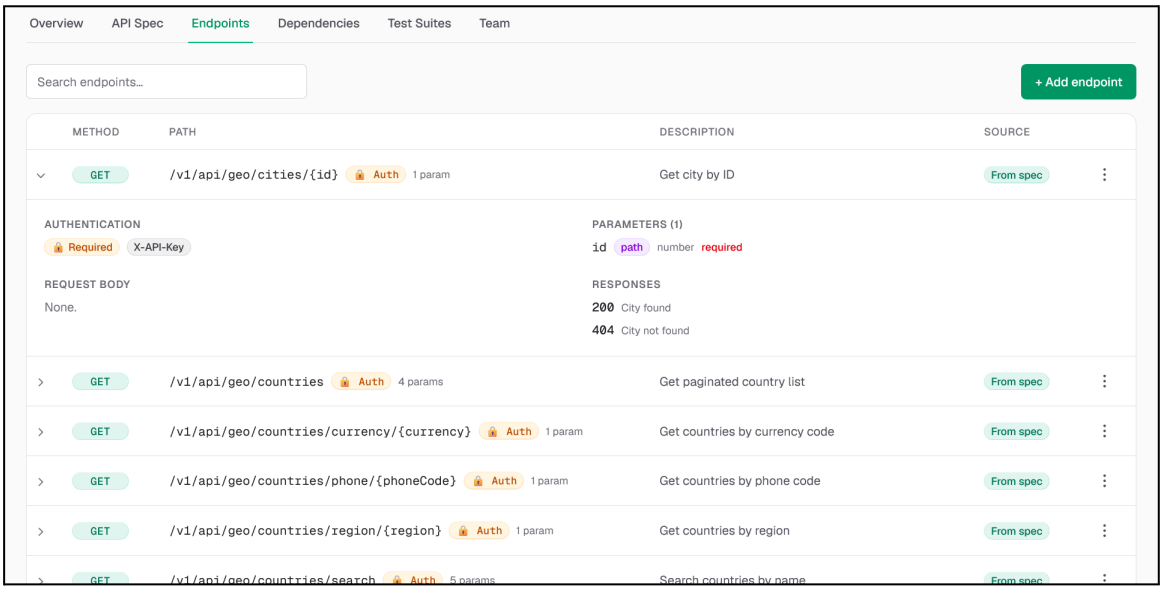


Figure 8.12: API Endpoints

Project owners and admins can add an endpoint manually with "+ Add endpoint"; choosing a Method, entering a Path, and optionally a Description for cases the specification does not cover. Manual endpoints can also be deleted from the row menu.

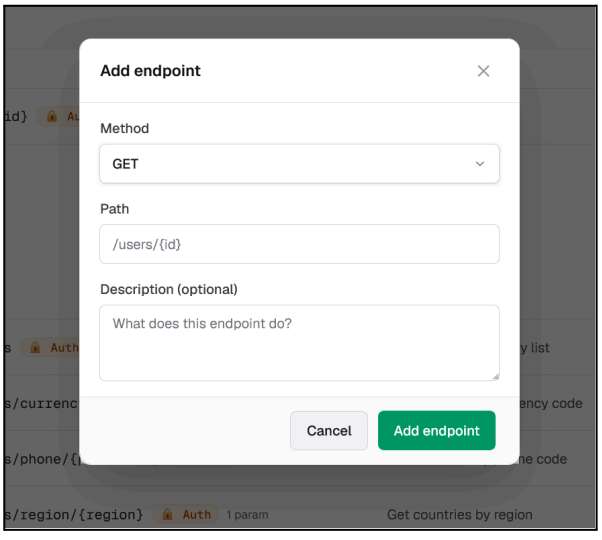


Figure 8.13: Add API Endpoint manually

8.12 Dependency Graph

The Dependencies tab visualizes the Semantic Property Dependency Graph the engine builds from the specification: an edge from one operation to another means the first

operation is believed to produce a value the second one needs; for example, an edge from "create a resource" to "fetch that resource by id" because the id one returns is what the other expects as input. Building the graph only reads the specification; it never sends any request to the API itself.

- Click "Build dependency graph" (available to admins and testers; viewers ask a teammate with one of those roles to do it). This takes roughly a minute, since the engine compares every operation against every other one.
- Once built, the graph can be rebuilt at any time from the same tab.

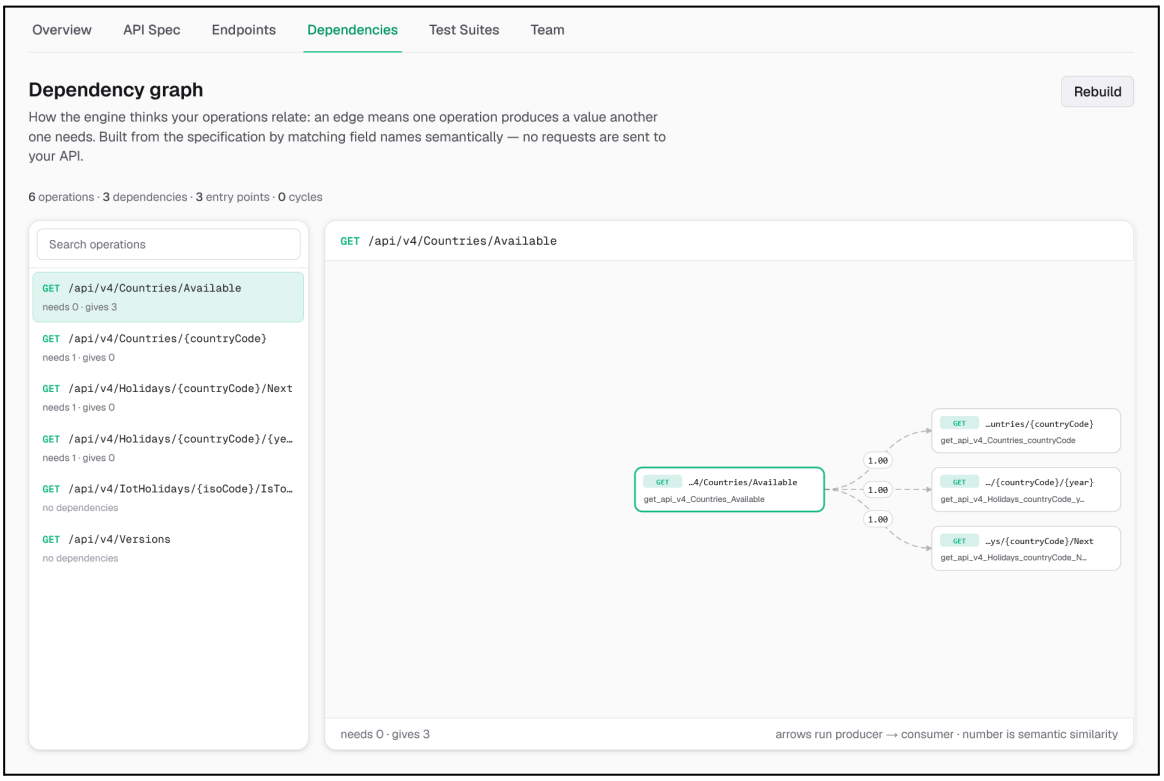


Figure 8.14: SPDG Visualization

8.13 Test Suites

The Test Suites tab is where AI-generated test runs are configured, executed, and reviewed. Each entry in its table is one run: its name (or a short generated id), status, target URL, pass/total result once complete, and creation date.

8.13.1 Creating a New Run

1. Click "+ New run".
2. Optionally give the run a Name.
3. Enter the Target URL; the live base URL the engine will send requests to.
4. Set the Time budget in seconds (default 300); how long the AI engine is allowed to spend generating and sending requests.
5. Optionally set a Mutation rate between 0 and 1, controlling how aggressively the engine varies generated values.
6. An "Advanced: Custom headers" section can be expanded to attach up to 20 HTTP headers to every request the run sends; for example an Authorization header for Basic or Bearer authentication, or an API-key header the target API

expects. Each row is a free-form key and value; common header names are suggested as you type.

7. Click "Create" to save the run.

A newly created run starts in the Pending state and has not sent any requests yet.

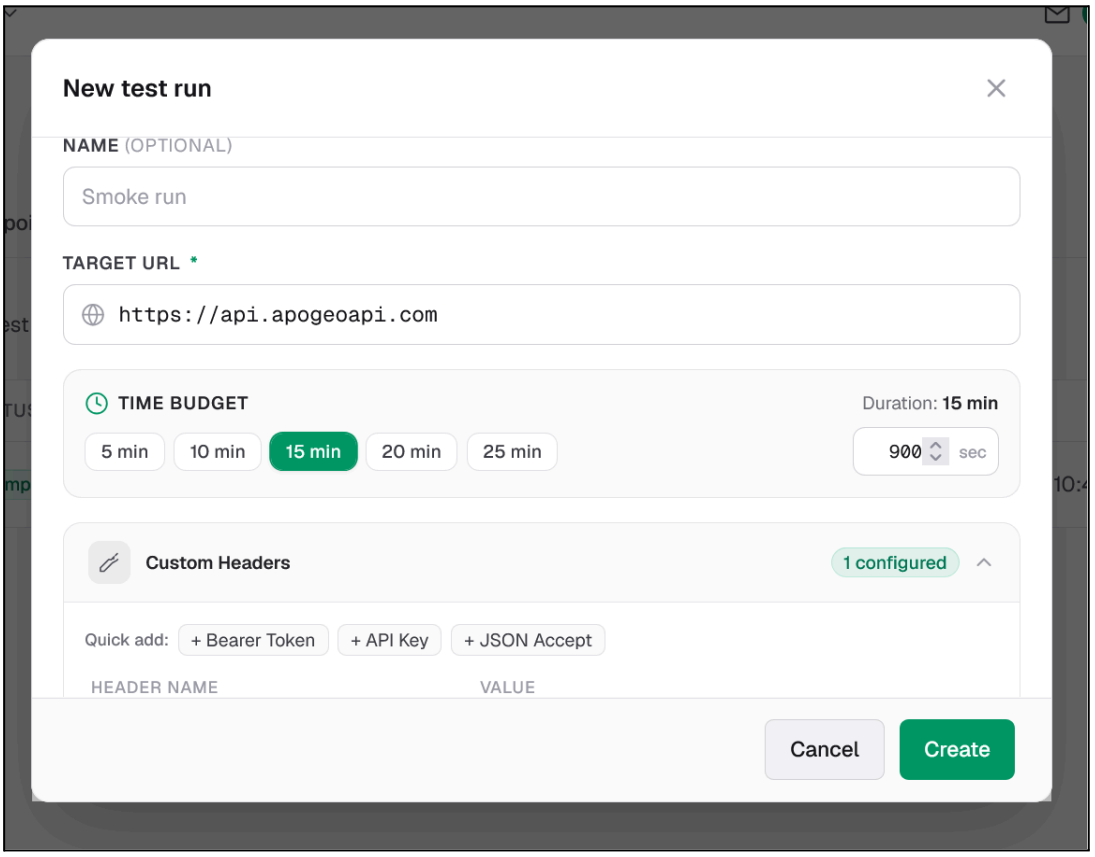


Figure 8.15: Create test suite

8.13.2 Running and Monitoring a Test Run

Open a pending run and click "Run tests" to start it. While running, the page polls automatically every few seconds and needs no manual refresh. A run can end in one of four states:

Status	Meaning
Pending	Configured, but not yet started.
Running	The AI engine is actively generating and sending requests to the target.
Completed	The run finished and produced a report.
Failed	The run could not be completed. For example because the target URL was unreachable or the engine service was unavailable. It can be retried.

6.13.3 Viewing the Test Report

Once a run completes, its page shows a full report. A run's job is finding real defects, not just checking for a non-2xx response: a request that receives a 4xx means the API correctly rejected a bad request, while a 5xx server error is the genuine fault signal. The headline banner reflects this directly, reporting the number of faults (5xx responses) detected, or confirming that none were found.

Below the banner:

- A KPI row shows coverage percentage, endpoints covered, total requests sent, and the breakdown of successful (2xx), client-error (4xx), and server-error (5xx) responses.
- A status-code distribution chart shows the same breakdown visually.
- A per-endpoint results table lists every endpoint the run exercised, with a Result badge (Successful / Client error / Server error), the exact status codes returned, and any failure explanation generated for it. It can be filtered by search term, outcome, or exact status code.
- For a completed run whose report has failures, an "Explain failures" button asks the AI engine to generate a plain-language explanation of what likely caused each failure and which request triggered it.
- The same dependency graph described in section 1.12 is shown again here, scoped to this run — which lets you see exactly which dependencies the reinforcement-learning agent confirmed while generating this particular run's requests.

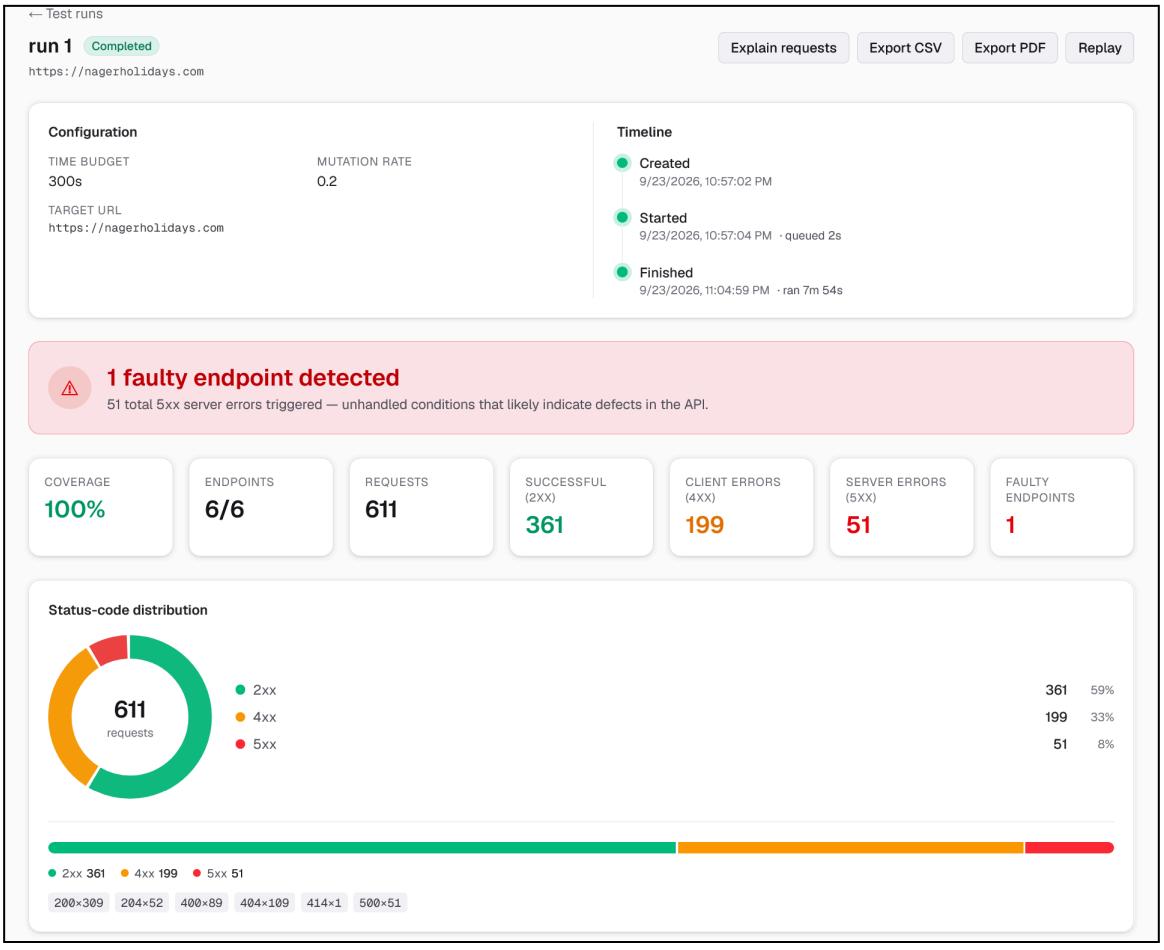


Figure 8.16: Test report

8.13.4 Exporting Reports

A completed run offers "Export CSV" and "Export PDF" buttons, each of which downloads the run's report in that format for sharing outside the application.

8.13.5 Replaying a Run

A completed run can be replayed with the "Replay" button: this resends that run's exact captured requests, in the same order, as a new, separate run — with no new AI generation involved. This is useful for confirming whether a fix actually resolved a previously-detected fault, since it is an apples-to-apples comparison against the same requests. A run's page shows a Run History panel listing its original run and every replay of it, so results can be compared side by side.

8.14 Captured Requests and Live Request Runs

From a completed run's report, clicking "View all captured requests" — or "View" next to a specific endpoint, or a node in the run's dependency graph — opens the full list of every request the engine actually sent during that run. The list can be filtered by endpoint and by response status class (2xx/3xx/4xx/5xx) or an exact status code, and is paginated 50 rows at a time.

Expanding a request shows the complete exchange: the request's method, URL, headers, and body on one side, and the response's status, duration, headers, and body on the other. A "Copy as curl" button builds a ready-to-run curl command from the captured request.

A "Run" button lets an admin or tester resend that exact request to the target API right now, and shows the fresh response inline as a Live response, without leaving the page. Because a non-GET/HEAD request can have real side effects on the target system, running anything other than a GET or HEAD request asks for confirmation first.

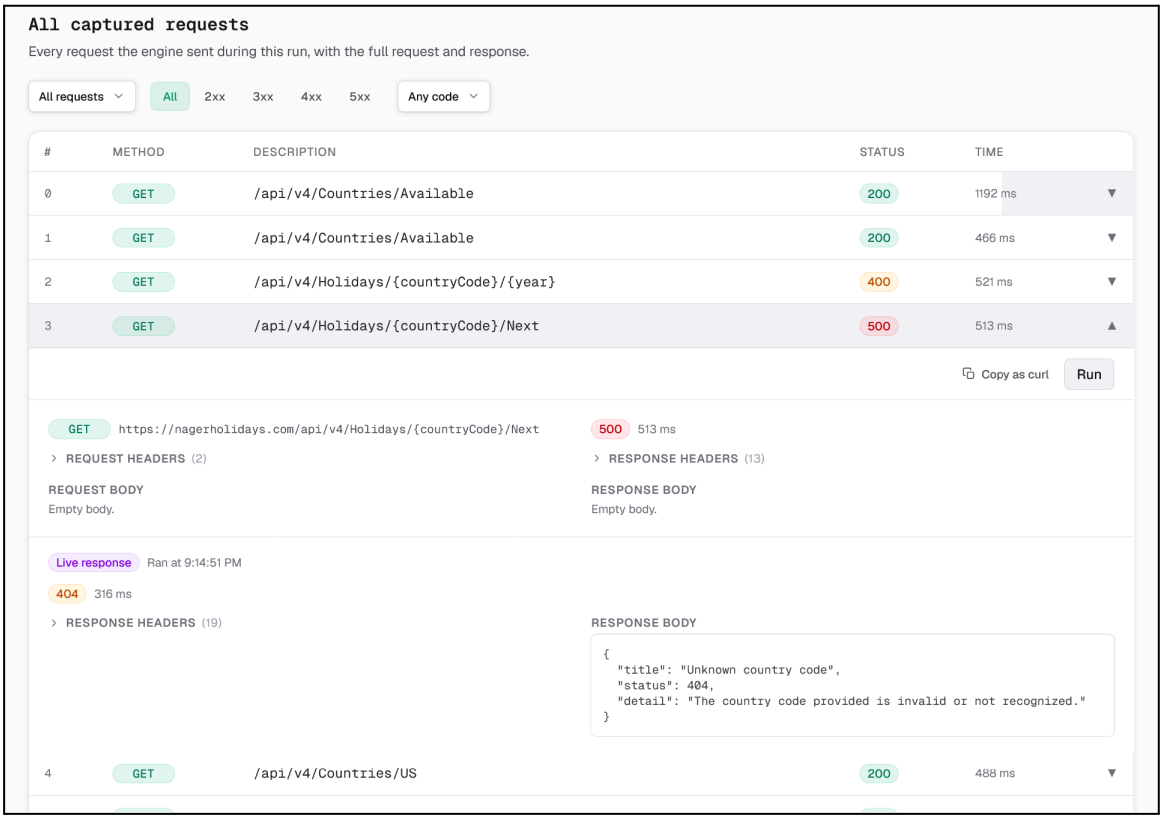


Figure 8.17: Test request log detail

8.15 Team Management

8.15.1 Managing Members

The Team tab (and the shorter team list on the Overview tab) shows everyone with access to the project: the owner, then every other member with their role. Owners and admins can change another member's role (Admin, Tester, or Viewer) directly from a dropdown next to their name, or remove them from the project with "Remove". Any member can leave a project they no longer need access to with "Leave".

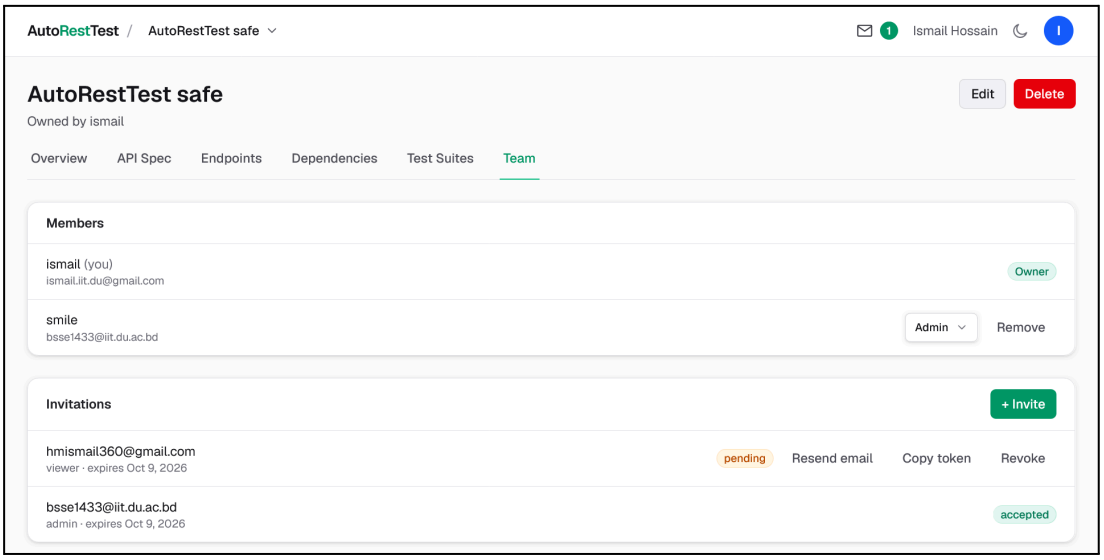


Figure 8.18: Team collaboration management

8.15.2 Inviting Team Members

1. On the Team tab, click "+ Invite" (owners and admins only).
2. Enter the teammate's Email address and choose a Role — Admin, Tester, or Viewer, each with a short description of what it allows.
3. Click "Send invite".

The invitation is emailed to that address. Pending invitations are listed on the Team tab with their role, expiry date, and status; each can be resent, have its invite token copied, or be revoked before it is accepted.

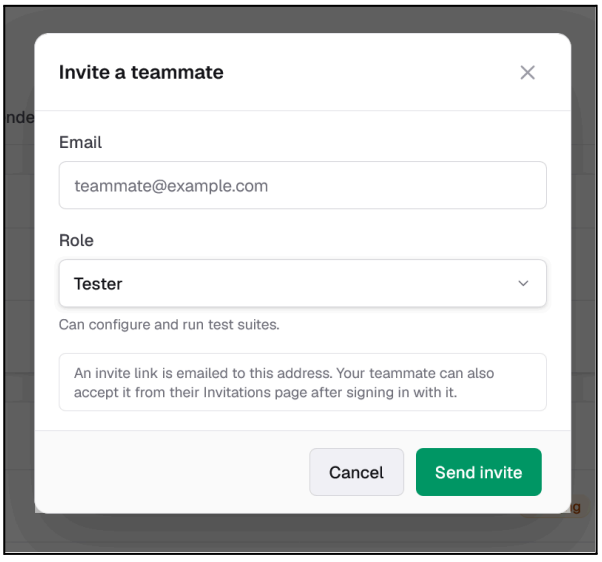


Figure 8.19: Add team member

8.15.3 My Invitations

Invitations addressed to the signed-in user's own account — from any project — are collected on the "Invitations" page, reached from the bell icon or account menu in the top bar. Each shows the project, the offered role, who sent it, and when it expires, with Accept and Decline buttons. Accepting takes you directly into the project.



Figure 8.20: My invitations page

8.16 Account Settings

The Settings page, reached from the account menu, is organized into four sections.

8.16.1 Profile

Lets you choose an avatar colour and set an optional Display name. Your Username and Email are shown read-only — the username is what you sign in with and cannot be changed, and every account on the platform is always verified, so no separate verification badge or step applies here. Click "Save changes" once you have made an edit.

8.16.2 Password

Lets you change your password by entering your Current password and a New password (entered twice to confirm). Changing your password signs you out of every device, including the one you used to change it — you will need to sign in again with the new password.

8.16.3 Notifications

Two checkboxes control optional email notifications and are saved the instant you toggle them: "Test run finished" (an email when a run you started completes or fails) and "Project invitations" (an email when someone invites you to a project — invitations still appear inside the application either way). Security emails — verification codes, password-reset links, and password-change confirmations — are always sent and cannot be turned off.

8.16.4 Deleting Your Account

The Danger Zone section offers "Delete my account". Since deleting your account also deletes every project you own — along with its specification, test runs, and captured requests, and removes your teammates' access to them — the confirmation dialog states exactly how many owned projects will be lost, and requires your current password before proceeding. This action cannot be undone.

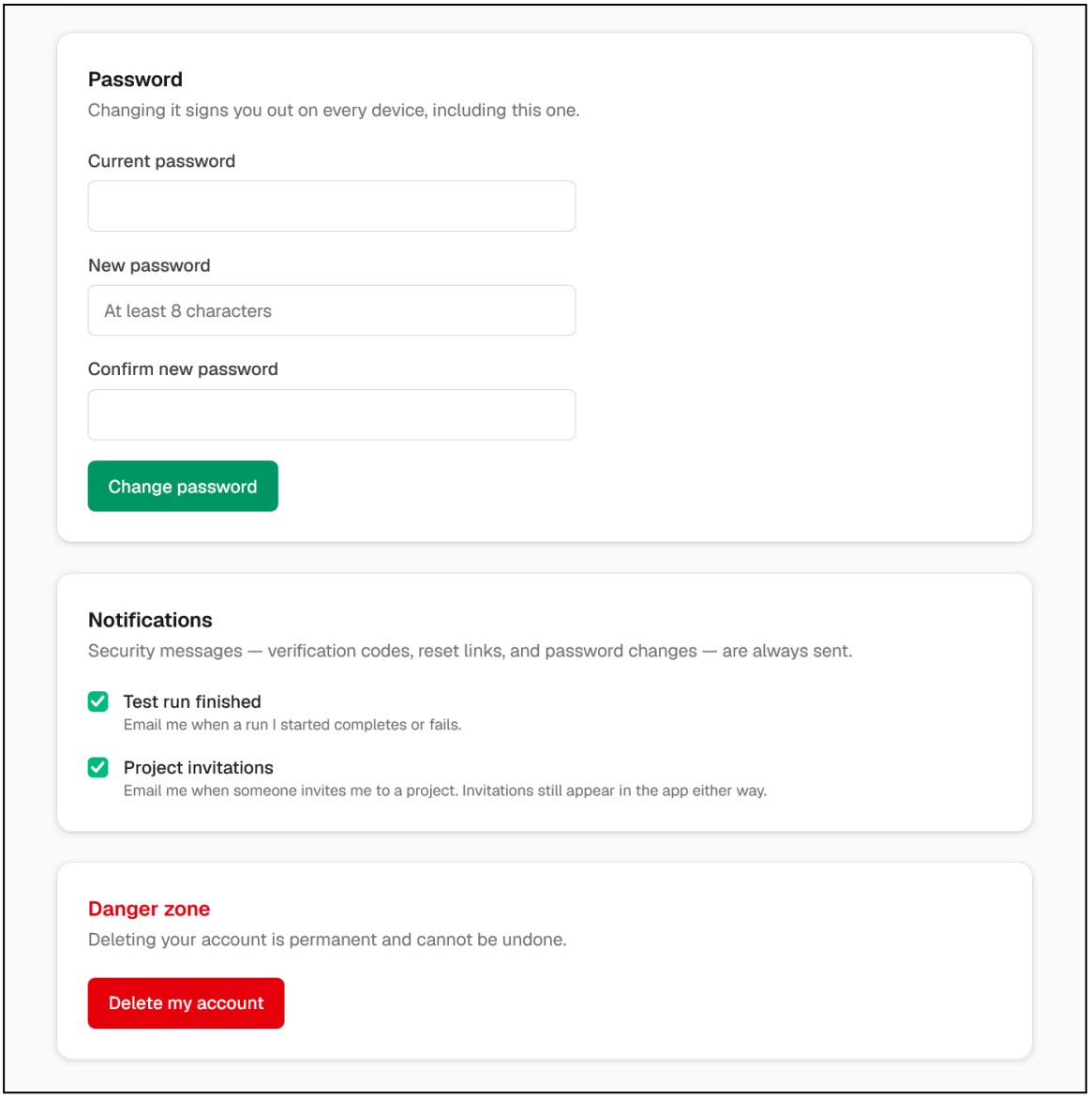


Figure 8.21: Account settings page

8.17 Logging Out

To end your session, open the account menu from your avatar in the top bar and click "Sign out". You are returned to the Login page.

8.18 Frequently Asked Questions

Q: I registered but never got my six-digit code. What should I do?
A: Use the "Resend the code" link on the verification screen — it is available again 60 seconds after the previous send. If the address was mistyped, use "Sign up again" to restart with the correct one.

Q: Why can't I change my username?
A: A username is the identity you sign in with, so AutoRestTest treats it as permanent once an account is created. You can still set a separate Display name in Settings, which is what teammates see around the app.

Q: Generating a specification from my source code is taking a very long time. Is something wrong?
A: Not necessarily. The pipeline reads your source code and calls an AI model to infer the specification, and large codebases can take a long time end-to-end. The page updates automatically, so it is safe to leave and come back later. Excluding vendored and build directories in the Exclude directories field before starting makes it noticeably faster.

Q: My completed run shows some 4xx responses. Is that a failure?
A: Not on its own. AutoRestTest treats a 4xx response as the API correctly rejecting a request it considered invalid — it is only counted as a fault when the API returns a 5xx server error, which indicates the API failed to handle the request at all.

Q: What is the difference between rebuilding a run and replaying it?
A: Re-running a pending run for the first time uses the AI engine to generate new requests. "Replay", available on a completed run, instead resends that exact same set of previously captured requests, in the same order, as a new run with no new generation — useful for confirming whether a fix actually resolved a fault.

Q: I accidentally deleted a project, a test run, or an endpoint. Can I get it back?
A: No. Every delete in AutoRestTest is permanent and cannot be undone, which is why each one asks for confirmation first. Make sure you mean to delete something before confirming.

Q: Can I invite someone who doesn't have an AutoRestTest account yet?
A: Yes. An invitation is sent to an email address, not an existing account; the recipient can register (or sign in, if they already have an account under that address) and then find the invitation waiting on their own "Invitations" page.

Q: Why is the "Run" button disabled on a captured request?
A: Resending a live request is limited to project Admins and Testers. Viewers can see every captured request but cannot trigger new ones against the live target API.

Chapter 9
Conclusion

AutoRestTest bridges the gap in REST API testing by transforming manual, fragmented test creation into an intelligent, fully automated process. By integrating Multi-Agent Reinforcement Learning, a Semantic Property Dependency Graph, and Large Language Models, the platform efficiently navigates complex, interdependent API operations.
Beyond simple test execution, the system actively accelerates debugging by providing plain-language, AI-generated explanations whenever it uncovers hidden server-side faults. Built as a collaborative SaaS platform, it allows teams to securely manage specifications, run customized test suites, and track historical results.
Ultimately, AutoRestTest empowers developers to maximize fault detection with minimal manual effort, ensuring highly reliable web services.

References

[1] Github Repository: https://github.com/CodeWithIsmail/AutoRestTest

[2] A Multi-Agent Approach for REST API Testing with Semantic Graphs and LLM-Driven Inputs, in Proceedings of 47th International Conference on Software Engineering (ICSE 2025)
Tyler Stennett, Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso,
Paper: https://arxiv.org/abs/2411.07098v2

[3] OOPS: Automated generation of REST API specification via LLMs
Hao Chen, Yunchun Li, Chen Chen, Fengxu Lin, Wei Li
Paper: https://arxiv.org/abs/2601.12735

[4] NVIDIA NIM API: https://build.nvidia.com/explore/discover